

Java | Python | Data Structures | SQL



SECTION 1: JAVA



PART A: Fundamentals

Q1. Explain the key principles of Object-Oriented Programming in Java.

Answer:

OOP is built on four core principles:

Principle	Meaning
Encapsulation	Wrapping data (variables) and methods inside a class; hiding internal details using access modifiers
Inheritance	A child class acquires properties and behaviors of a parent class using extends
Polymorphism	One interface, many implementations — same method behaves differently based on context
Abstraction	Hiding complex implementation and showing only essential features to the user

Real-World Example: Think of a **Bank Account system**:

- **Encapsulation:** **balance** is private; you access it only via **getBalance()** and **deposit()** methods
- **Inheritance:** **SavingsAccount** and **CurrentAccount** extend the base **BankAccount** class
- **Polymorphism:** **calculateInterest()** method works differently for savings vs. current accounts
- **Abstraction:** You use an ATM interface without knowing how cash dispensing works internally

Q2. What is the difference between JDK, JRE, and JVM?

Answer:

Component	Full Form	Purpose
JVM	Java Virtual Machine	Executes Java bytecode — the runtime engine
JRE	Java Runtime Environment	JVM + libraries needed to <i>run</i> Java programs
JDK	Java Development Kit	JRE + compiler (javac) + tools needed to <i>develop</i> Java programs

Hierarchy: JDK $\supset$ JRE $\supset$ JVM

- **JVM:** The engine that converts bytecode to machine-specific instructions
- **JRE:** What an end user needs to run a Java application
- **JDK:** What a developer needs to write, compile, and run Java code

Real-World Example: Think of building a car (Java program):

- **JDK** = Full factory (tools + assembly line + test track) — for the developer
- **JRE** = Test track + fuel — enough to drive/run the car
- **JVM** = The engine inside — what actually makes the car move on *that specific road* (OS)

A user who only wants to run your app needs the **JRE**. You, the developer, need the **JDK**.

Q3. Explain "Write Once, Run Anywhere" in Java.

Answer:

Java source code (`.java`) is compiled by `javac` into **bytecode** (`.class` files) — not into machine-specific binary. This bytecode is **platform-neutral**. The **JVM** on each operating system (Windows, Linux, Mac) reads and executes this same bytecode, translating it into machine-specific instructions at runtime.

Steps:

1. Developer writes `Hello.java` on Windows
2. `javac Hello.java` → produces `Hello.class` (bytecode)
3. `Hello.class` runs on Windows JVM, Linux JVM, or Mac JVM — without recompiling

Real-World Example: A banking application built in Java on a Windows development machine can be deployed to a Linux server at the data center and accessed on a Mac by the bank manager — all running the same `.class` files. No code change needed. This is why Java dominated enterprise software in the 2000s — one codebase for all platforms.

Q4. What is the difference between primitive types and wrapper classes?

Answer:

Feature	Primitive Types	Wrapper Classes
Examples	int, char, boolean, double	Integer, Character, Boolean, Double
Stored in	Stack (by value)	Heap (as objects)
Default value	0, false, '\u0000'	null
Used in Collections	✗ Cannot use directly	✓ Required
Utility methods	None	<code>Integer.parseInt()</code> , <code>Double.valueOf()</code> , etc.
Autoboxing	Primitive → Wrapper automatically	Unboxing: Wrapper → Primitive automatically

Real-World Example: You're building a student grade system using an `ArrayList`. You can't write `ArrayList<int>` — Java generics only work with objects. So you use `ArrayList<Integer>`. Java **autoboxes** automatically: when you add `95` (an `int`), Java silently converts it to `new Integer(95)`. When you retrieve it and compare, it **unboxes** back to `int`. This happens seamlessly behind the scenes.

Q5. Explain pass-by-value in Java. How are objects passed?

Answer:

Java is **strictly pass-by-value** — always. But what is the "value" being passed?

- For **primitives**: The actual value is copied. Changing the parameter inside the method does NOT affect the original.
- For **objects**: The *reference* (memory address) is copied — not the object itself. Changes to the object's internal state WILL be visible outside, but reassigning the reference will NOT affect the original variable.

```
// Primitive: original unchanged
void increment(int x) { x++; } // x is a copy

// Object: internal state changes visible
void changeName(Person p) { p.name = "Alice"; } // p points to same object
void reassign(Person p) { p = new Person(); }    // original reference unchanged
```

Real-World Example: Imagine you share a **Google Doc link** with a friend (passing object reference). If they edit the document's content — you both see changes (modifying object state). But if they click "Make a copy" and start editing the copy — your original doc is unaffected (reassigning the reference). Java works exactly this way with objects.

Q6. What are access modifiers in Java? (public, private, protected, default)

Answer:

Access modifiers control **visibility** of classes, methods, and variables.

Modifier	Same Class	Same Package	Subclass	Everywhere
private	✓	✗	✗	✗
default (no keyword)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Best Practice: Follow the **principle of least privilege** — expose only what is necessary.

Real-World Example: In a hospital management system:

- `private double salary` — only the `Employee` class can access it (no outsider can see your salary)

- `protected String department` — subclasses like `Doctor` and `Nurse` can use it
- `default String hospitalName` — only classes in the same package (same hospital module) can see it
- `public String getName()` — any part of the system can get the employee's name

PART B: Memory & OOP

Q7. Explain Java's memory model (heap, stack, method area).

Answer:

Java divides runtime memory into several areas:

Area	What it stores	Lifetime
Stack	Local variables, method calls, primitive values	Per method call — freed when method returns
Heap	All objects and arrays (created with <code>new</code>)	Until garbage collected
Method Area	Class metadata, static variables, bytecode	Entire program lifetime
PC Register	Current instruction being executed	Per thread
Native Method Stack	Native (non-Java) method calls	Per thread

Real-World Example: When a bank's `transfer()` method runs:

- **Stack:** Holds local variables like `amount`, `fromAccountId` — created when method starts, destroyed when it ends
- **Heap:** Holds the actual `Account` objects (`fromAccount`, `toAccount`) — persists beyond the method
- **Method Area:** Holds `Account.class` metadata and any `static double interestRate` — loaded once when the class is first used

Q8. How does garbage collection work in Java?

Answer:

Java's **Garbage Collector (GC)** automatically reclaims memory occupied by objects that are no longer reachable (no references pointing to them). Developers don't manually free memory — the JVM handles it.

Key Concepts:

- **Generational GC:** Heap is divided into Young Generation, Old Generation, and Metaspace
- **Young Gen:** New objects go here. Most die young → collected by **Minor GC** (fast)
- **Old Gen:** Long-surviving objects promoted here → collected by **Major/Full GC** (slow)
- **GC Roots:** Objects reachable from stack variables, static fields, or JNI references are "alive"

Common GC Algorithms: Serial, Parallel, G1 (default since Java 9), ZGC

Real-World Example: An e-commerce app processes 1000 orders/minute. Each **Order** object is created, processed, and then no variable holds a reference to it. The GC detects these unreferenced **Order** objects (unreachable) in the Young Generation and sweeps them away during Minor GC — happening in milliseconds without the developer writing any **delete** or **free** statements.

Q9. What is the difference between `==` and `.equals()`?

Answer:

- `==` compares **references** (memory addresses) for objects — checks if both variables point to the *same object in heap*. For primitives, it compares actual values.
- `.equals()` compares **content/logical equality** — checks if two objects are *meaningfully equal* based on their state. Classes override this method to define custom equality.

```
String a = new String("hello");
String b = new String("hello");

System.out.println(a == b);           // false - different objects in heap
System.out.println(a.equals(b));      // true  - same content
```

⚠ **String Pool Exception:** String literals are cached, so `"hello" == "hello"` is true (same pool reference) — but rely on `.equals()` always.

Real-World Example: Two different **Customer** objects created from the same data: `Customer c1 = new Customer("Alice", 101)` and `Customer c2 = new Customer("Alice", 101)`. `c1 == c2` is false (different heap locations). By overriding `.equals()` to compare `customerId`, `c1.equals(c2)` returns **true** — critical for deduplication logic in a CRM system.

Q10. Explain method overloading vs method overriding.

Answer:

Feature	Overloading	Overriding
Where	Same class	Parent & child class
Signature	Same name, different parameters	Identical name and parameters
Return type	Can differ	Must be same (or covariant)
Resolved at	Compile time (Static Polymorphism)	Runtime (Dynamic Polymorphism)
Annotations	None required	<code>@Override</code> recommended
Access modifier	Can change freely	Cannot be more restrictive

Real-World Example: In a **Payment Gateway**:

- **Overloading:** `processPayment(double amount)`, `processPayment(double amount, String currency)`, `processPayment(double amount, String currency, String promoCode)` — same method name, different parameters for different use cases
- **Overriding:** `PaymentGateway` base class has `authenticate()`. `PayPalGateway` and `StripeGateway` both override it with their specific authentication logic. At runtime, Java calls the right version based on the actual object type.

Q11. What is polymorphism? Provide examples.

Answer:

Polymorphism means "many forms" — the ability of one reference to behave differently based on the actual object it points to.

Two types:

1. **Compile-time (Static) Polymorphism:** Method Overloading — resolved at compile time
2. **Runtime (Dynamic) Polymorphism:** Method Overriding — resolved at runtime via dynamic method dispatch

```
// Runtime polymorphism
Animal a = new Dog();    // Animal reference, Dog object
a.makeSound();           // calls Dog's makeSound(), not Animal's — decided at
runtime
```

Real-World Example: A Notification System at a company:

```
Notification n;
n = new EmailNotification();    → n.send() sends an email
n = new SMSNotification();      → n.send() sends an SMS
n = new PushNotification();     → n.send() sends a push alert
```

Same method call `n.send()` — three different behaviors based on the actual object. You can write a loop that processes all notifications uniformly without if-else chains. This is the power of polymorphism.

PART C: Collections & Data Structures

Q12. Compare ArrayList vs LinkedList. When would you use each?

Answer:

Feature	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked list

Feature	ArrayList	LinkedList
Access by index	$O(1)$ — fast	$O(n)$ — slow (must traverse)
Insert/Delete at middle	$O(n)$ — elements shift	$O(1)$ — just change pointers
Insert at end	$O(1)$ amortized	$O(1)$
Memory	Less (array)	More (each node stores prev/next pointers)
Implements	List	List, Deque, Queue

Use ArrayList when: Frequent reads/access by index, rare insertions in the middle. **Use LinkedList when:** Frequent insertions/deletions at front or middle, using as a Queue or Deque.

Real-World Example:

- **ArrayList:** Displaying a product catalog on an e-commerce site — you frequently access products by index to display page 2, 3, etc.
- **LinkedList:** A music playlist where users constantly add songs at the current position or remove songs — $O(1)$ insertions make it ideal. Spotify-style "add next in queue" is a perfect LinkedList use case.

Q13. Explain the internal working of HashMap.

Answer:

HashMap stores **key-value pairs** using an **array of buckets** (internally called `Node[]`). Here's how it works:

1. **Put:** `hashCode()` of the key is computed → compressed to array index → key-value stored in that bucket
2. **Collision:** If two keys hash to the same bucket → stored as a **linked list** at that bucket (Java 8+: becomes a **Red-Black Tree** if chain length > 8)
3. **Get:** Same hash computation → find bucket → traverse list/tree → compare with `.equals()` → return value
4. **Load Factor (default 0.75):** When 75% full → **rehashing** doubles the array size, redistributes entries
5. **Initial Capacity:** Default 16 buckets

Real-World Example: A phone contacts app stores names (keys) and phone numbers (values) in a HashMap. When you type "Alice," the app computes Alice's hash, goes directly to the right bucket, and retrieves the number in $O(1)$ — instead of scanning all 500 contacts. If two names happen to hash to the same bucket (collision), they're stored in a chain and resolved using `.equals()`.

Q14. What is the difference between HashMap and Hashtable?

Answer:

Feature	HashMap	Hashtable
Thread safety	✗ Not synchronized	✓ Synchronized (thread-safe)
Performance	Faster (no lock overhead)	Slower

Feature	HashMap	Hashtable
Null keys/values	✅ One null key, many null values	❌ No null keys or values
Since	Java 1.2	Java 1.0 (legacy)
Preferred alternative	ConcurrentHashMap for thread safety	Avoid — outdated
Iteration	Iterator (fail-fast)	Enumeration (not fail-fast)

Real-World Example: An old banking system from the 1990s used **Hashtable** because multiple threads accessed customer data simultaneously and it was thread-safe. A modern rewrite uses **HashMap** (faster) for single-threaded operations, and **ConcurrentHashMap** where multiple threads update accounts — it's faster than Hashtable because it locks only *segments* of the map, not the whole thing.

Q15. Compare HashSet, LinkedHashSet, and TreeSet.

Answer:

All implement the **Set** interface (no duplicates), but differ in ordering and performance:

Feature	HashSet	LinkedHashSet	TreeSet
Ordering	No guaranteed order	Insertion order	Sorted (natural or Comparator)
Null element	✅ One null	✅ One null	❌ No null
Performance	O(1) add/remove/contains	O(1) slightly slower	O(log n) all operations
Internal structure	HashMap	LinkedHashMap	Red-Black Tree
Use when	Fast lookup, no order needed	Order of insertion matters	Sorted data needed

Real-World Example:

- **HashSet:** Checking if a user's email is already registered — just need fast lookup
- **LinkedHashSet:** Displaying a user's recently searched keywords in the order they searched them, no duplicates
- **TreeSet:** Maintaining a **leaderboard** of unique usernames sorted alphabetically — TreeSet automatically keeps them in order as new players join

Q16. What is the Collections Framework in Java?

Answer:

The **Java Collections Framework (JCF)** is a unified architecture for storing and manipulating groups of objects. It provides:

- **Interfaces:** **Collection**, **List**, **Set**, **Queue**, **Map**, **Deque**

- **Implementations:** `ArrayList`, `LinkedList`, `HashMap`, `TreeSet`, `PriorityQueue`, etc.
- **Algorithms:** `Collections.sort()`, `Collections.shuffle()`, `Collections.binarySearch()`, etc.

Key Hierarchy:

```
Collection
├── List → ArrayList, LinkedList, Vector
├── Set  → HashSet, LinkedHashSet, TreeSet
├── Queue → PriorityQueue, LinkedList
Map (separate hierarchy)
├── HashMap, TreeMap, LinkedHashMap, Hashtable
```

Real-World Example: A food delivery app's backend uses the entire Collections Framework together:

- `HashMap<String, Restaurant>` → look up restaurant by name
- `PriorityQueue<Order>` → process orders by urgency
- `ArrayList<MenuItem>` → display restaurant menus
- `HashSet<String>` → store unique cuisine types All managed by the same framework with consistent APIs.

PART D: Advanced

Q17. What are generics in Java? Why use them?

Answer:

Generics allow you to write **type-safe, reusable code** by parameterizing types. Instead of working with `Object` (requiring casting), you specify the type at compile time.

```
// Without generics – unsafe, requires casting
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0); // ClassCastException risk

// With generics – type-safe
List<String> list = new ArrayList<>();
list.add("hello");
String s = list.get(0); // no cast needed
```

Benefits:

- Compile-time type checking — catches errors early
- Eliminates unnecessary casting
- Enables writing generic algorithms (works for any type)

Real-World Example: A hospital's inventory system stores different items: medicines, equipment, consumables. A generic `Repository<T>` class with `save(T item)` and `findById(int id)` works for `Repository<Medicine>`, `Repository<Equipment>` — one implementation, type-safe for all. Without generics, you'd need separate classes or cast from `Object` — error-prone.

Q18. Explain lambda expressions in Java 8+.

Answer:

Lambda expressions are concise, anonymous functions that can be passed as arguments or returned from methods. They enable **functional programming** style in Java and are primarily used with **functional interfaces** (interfaces with exactly one abstract method).

Syntax: `(parameters) -> expression` or `(parameters) -> { statements; }`

```
// Before Java 8 – anonymous inner class
Runnable r = new Runnable() {
    public void run() { System.out.println("Running"); }
};

// With lambda – clean and concise
Runnable r = () -> System.out.println("Running");

// Sorting with lambda
list.sort((a, b) -> a.compareTo(b));
```

Real-World Example: An e-commerce platform needs to filter, sort, and transform product lists differently for different categories. With lambdas: `products.stream().filter(p -> p.getPrice() < 500).sorted((a, b) -> a.getRating() - b.getRating()).collect(toList())` — reads like plain English and replaces 20+ lines of anonymous class boilerplate. Lambda expressions made Java code dramatically cleaner for data processing pipelines.

Q19. What is the Stream API? How does it differ from loops?

Answer:

The **Stream API** (Java 8+) provides a functional, declarative way to process sequences of elements. Unlike loops, streams describe **what** to do, not **how** to do it.

Feature	Traditional Loop	Stream API
Style	Imperative (how)	Declarative (what)
Mutability	Modifies external variables	Produces new collections
Parallel	Manual thread management	<code>parallelStream()</code> — automatic
Readability	Can be verbose	Concise, chainable

Feature	Traditional Loop	Stream API
Lazy evaluation	No	Yes — operations run only when needed

```
// Loop approach
List<String> result = new ArrayList<>();
for (Employee e : employees) {
    if (e.getSalary() > 50000) result.add(e.getName().toUpperCase());
}

// Stream approach – same result
List<String> result = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .map(e -> e.getName().toUpperCase())
    .collect(Collectors.toList());
```

Real-World Example: A payroll system processes 10,000 employees: filter by department, calculate tax, sum total payroll. With `parallelStream()`, the work splits across CPU cores automatically — what takes 2 seconds with a loop takes 0.4 seconds with parallel streams. Netflix uses Java streams internally for processing massive content recommendation pipelines.

Q20. Explain exception handling in Java (try-catch-finally).

Answer:

Java uses a structured exception handling mechanism to manage runtime errors gracefully without crashing the program.

Key Blocks:

- **try** — Code that might throw an exception
- **catch** — Handles the specific exception
- **finally** — Always executes (cleanup: close files, connections) — even if exception occurs or is not caught
- **throw** — Manually throw an exception
- **throws** — Declare exceptions a method might throw

Exception Hierarchy:

```
Throwable
├── Error (JVM errors – don't catch: OutOfMemoryError)
├── Exception
│   ├── Checked (must handle: IOException, SQLException)
│   └── Unchecked/Runtime Exception (optional: NullPointerException,
        ArrayIndexOutOfBoundsException)
```

```
try {  
    Connection conn = DriverManager.getConnection(url);  
    // database operations  
} catch (SQLException e) {  
    System.err.println("DB error: " + e.getMessage());  
} finally {  
    conn.close(); // always close connection  
}
```

Real-World Example: A banking application transfers money between accounts. During the operation, a network timeout occurs (IOException). The `catch` block logs the error and rolls back the transaction (preventing data corruption). The `finally` block closes the database connection — regardless of success or failure — preventing connection leaks that would crash the server under heavy load.



SECTION 2: PYTHON



PART A: Fundamentals

Q21. Explain the Zen of Python and its philosophy.

Answer:

The **Zen of Python** (PEP 20) is a collection of 19 guiding aphorisms for writing good Python code. Access it by typing `import this` in Python. Key principles:

- *Beautiful is better than ugly* — Code aesthetics matter
- *Explicit is better than implicit* — Be clear, not clever
- *Simple is better than complex* — Prefer simplicity
- *Readability counts* — Code is read more than it's written
- *There should be one obvious way to do it* — One right approach
- *Errors should never pass silently* — Don't hide problems
- *In the face of ambiguity, refuse the temptation to guess*

Real-World Example: A junior developer writes a one-liner to reverse a list with a complex lambda expression just to be "clever." A senior Pythonista replaces it with `my_list[::-1]` — or even `list(reversed(my_list))` for clarity. In a data science team at Google, code review often rejects overly clever solutions in favor of readable ones — because six months later, the same developer won't understand their own "smart" code.

Q22. What makes Python an interpreted language?

Answer:

Python is **interpreted** because source code (`.py`) is executed **line by line** by the Python interpreter at runtime — not compiled to machine code ahead of time.

Process:

1. Python source code → **CPython** compiles to **bytecode** (`.pyc` files) — automatically
2. The **Python Virtual Machine (PVM)** interprets this bytecode line by line
3. Each line is translated to machine instructions and executed immediately

Implications:

- ☒ No separate compilation step — run directly with `python script.py`
- ☒ Platform independent (like Java bytecode)
- ☒ Dynamic typing — types checked at runtime
- ☒ Slower than compiled languages (C, C++) for CPU-intensive tasks
- ☒ Easier debugging — errors show the exact line

Real-World Example: A data scientist writes a Python script to analyze COVID data. They run it immediately — no compilation wait. When there's a bug on line 47, the interpreter stops exactly there with a clear error message, unlike C++ where linker errors can be cryptic. This rapid feedback loop is why Python dominates research environments like Jupyter Notebooks.

Q23. What is the Global Interpreter Lock (GIL)? How does it affect performance?

Answer:

The **GIL (Global Interpreter Lock)** is a mutex in CPython that allows only **one thread to execute Python bytecode at a time** — even on multi-core processors.

Why it exists: CPython's memory management (reference counting) is not thread-safe. The GIL protects it from race conditions.

Impact:

Scenario	Effect of GIL
CPU-bound tasks (math, data processing)	Multi-threading gives NO speedup — only one core used
I/O-bound tasks (file read, network, database)	Minimal impact — GIL released during I/O waits
Multi-processing	No GIL issue — separate processes, separate interpreters

Solutions for CPU-bound parallel work:

- Use `multiprocessing` module (separate processes)
- Use NumPy/Pandas (C extensions release the GIL)
- Use alternative interpreters: PyPy, Jython

Real-World Example: A web scraper fetching 1000 URLs is **I/O-bound** — threads work fine because the GIL is released while waiting for network responses (near-linear speedup with **threading**). But a video transcoder processing frames is **CPU-bound** — threads give no speedup. The fix: use **multiprocessing** to spawn 8 processes (one per CPU core), achieving true parallelism.

Q24. Compare lists, tuples, and sets in Python.

Answer:

Feature	List	Tuple	Set
Syntax	<code>[1, 2, 3]</code>	<code>(1, 2, 3)</code>	<code>{1, 2, 3}</code>
Mutability	Mutable	Immutable	Mutable
Ordered	✓ Yes	✓ Yes	✗ No
Duplicates	✓ Allowed	✓ Allowed	✗ Not allowed
Indexing	✓ Yes	✓ Yes	✗ No
Hashable (usable as dict key)	✗ No	✓ Yes	✗ No
Use when	Ordered, changeable collection	Fixed data, dict keys	Unique items, fast lookup

Real-World Example: In a student database system:

- **List:** `enrolled_courses = ["Math", "Physics", "CS"]` — courses can be added/dropped
- **Tuple:** `student_id = (name, dob, roll_no)` — identity fields never change; safer as a dict key
- **Set:** `unique_departments = {"CS", "Math", "Physics"}` — ensures no duplicate department names; fast O(1) membership check: `"Math" in unique_departments`

Q25. How do dictionaries work in Python? What can be keys?

Answer:

Python dictionaries are **hash tables** internally — they store key-value pairs and provide O(1) average-time lookups. Python 3.7+ guarantees **insertion order** is maintained.

What can be a key? Any **hashable** (immutable) object:

- ✓ Strings, integers, floats, tuples of hashables, frozensets
- ✗ Lists, dictionaries, sets (mutable — not hashable)

Internal mechanism:

1. `hash(key)` computed → determines bucket index
2. Key-value stored in that bucket
3. Collision resolved via **open addressing** (Python's approach)

```
student = {"name": "Alice", "gpa": 3.9, (2024, "Spring"): "semester_key"}
# tuple as key ✅ – immutable, hashable
```

Real-World Example: A university's grade lookup system: `grades = {"CS101": "A", "MATH201": "B+", "PHY301": "A-"}`. Retrieving `grades["CS101"]` is $O(1)$ regardless of 10 or 10,000 courses — hash directly to the answer. Using a list of tuples for the same purpose would require $O(n)$ linear search. Dictionary's hash table design makes it the backbone of Python's entire namespace system.

Q26. Explain mutable vs immutable objects.

Answer:

- **Mutable objects:** Can be changed after creation. The object's content in memory can be modified.
 - Examples: `list`, `dict`, `set`, custom class instances
- **Immutable objects:** Cannot be changed after creation. Any "modification" creates a new object.
 - Examples: `int`, `float`, `str`, `tuple`, `frozenset`, `bool`

Why it matters:

- Immutable objects are **thread-safe** (can't be accidentally changed)
- Immutable objects are **hashable** (can be dict keys or set members)
- Mutable objects passed to functions **can be modified** inside the function (side effects)

```
# Immutable – s is rebound to new object
s = "hello"
s += " world" # new string created, "hello" unchanged

# Mutable – modified in place
lst = [1, 2, 3]
lst.append(4) # same list object changed
```

Real-World Example: A configuration system uses a **tuple** for database connection settings `("localhost", 5432, "mydb")` — immutable, safe to pass anywhere without risk of accidental modification. Meanwhile, the **list** of active user sessions is mutable — users join and leave constantly, so you need to add/remove elements in place without creating new lists each time.

PART B: OOP & Functions

Q27. Explain classes and objects in Python.

Answer:

- **Class:** A blueprint or template that defines attributes (data) and methods (behavior)
- **Object (Instance):** A specific realization of a class with its own data

```
class Car:
    # Class attribute (shared by all instances)
    wheels = 4

    def __init__(self, brand, speed): # Constructor
        self.brand = brand           # Instance attribute
        self.speed = speed

    def accelerate(self):
        self.speed += 10
        return f"{self.brand} now at {self.speed} km/h"

my_car = Car("Toyota", 60) # Object created
print(my_car.accelerate()) # Toyota now at 70 km/h
```

Real-World Example: A ride-sharing app like Uber defines a **Driver** class with attributes **name**, **rating**, **location** and methods **accept_ride()**, **complete_trip()**. Each driver on the platform is a separate **object** — 100,000 drivers means 100,000 instances, each with their own location and rating, all sharing the same class blueprint and methods.

Q28. What is the difference between `__init__` and `__new__`?

Answer:

	<code>__new__</code>	<code>__init__</code>
Purpose	Creates the object (allocates memory)	Initializes the object (sets attributes)
Called	Before <code>__init__</code>	After <code>__new__</code> returns the instance
First argument	<code>cls</code> (the class)	<code>self</code> (the newly created instance)
Returns	The new instance	Nothing (<code>None</code>)
Override when	Creating immutable types, Singleton pattern, metaclasses	Almost always — for standard initialization

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton()
b = Singleton()
print(a is b) # True - same object
```

Real-World Example: A database connection manager uses `__new__` to implement the **Singleton pattern** — ensuring only one connection pool object ever exists across the entire application. `__new__` checks if an instance already exists; if yes, returns that same instance. `__init__` then (re)initializes its configuration. This prevents multiple expensive connection pools from being accidentally created.

Q29. Explain class methods, static methods, and instance methods.

Answer:

	Instance Method	Class Method	Static Method
Decorator	None	<code>@classmethod</code>	<code>@staticmethod</code>
First param	<code>self</code> (instance)	<code>cls</code> (class)	None
Access	Instance + class data	Class data only	No class/instance data
Called on	Object	Class or Object	Class or Object
Use when	Working with instance state	Factory methods, alternative constructors	Utility/helper functions

```
class Employee:
    company = "TechCorp"

    def __init__(self, name, salary):
        self.name = name          # instance method uses self
        self.salary = salary

    @classmethod
    def from_string(cls, emp_str): # alternative constructor
        name, salary = emp_str.split(",")
        return cls(name, float(salary))

    @staticmethod
    def is_valid_salary(salary):    # utility, no class/instance needed
        return 0 < salary < 1_000_000
```

Real-World Example: A `User` class in a web app: `login(self)` is an instance method (needs this specific user's data). `from_oauth_token(cls, token)` is a class method — an alternative constructor to create a User from a Google OAuth token. `validate_email(email)` is a static method — pure utility, no user object needed, can be called as `User.validate_email("test@gmail.com")`.

Q30. What are decorators in Python? How do they work?

Answer:

A **decorator** is a function that **wraps another function** to extend or modify its behavior — without changing the original function's code. It's Python's implementation of the Decorator design pattern.

Mechanics: A decorator takes a function as input, defines a wrapper function that adds behavior, and returns the wrapper.

```
def timer(func):
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.2f}s")
        return result
    return wrapper

@timer                                     # equivalent to: process_data = timer(process_data)
def process_data(n):
    return sum(range(n))

process_data(1_000_000)                   # prints: process_data took 0.03s
```

Real-World Example: A Django or Flask web app uses `@login_required` decorator on every route that needs authentication. Instead of writing "check if user is logged in" at the top of 50 different view functions, one decorator handles it universally. `@cache(timeout=300)` is another common decorator that caches expensive database query results for 5 minutes — applied with a single line above any function.

Q31. Explain lambda functions and their limitations.

Answer:

Lambda functions are small, anonymous (unnamed), single-expression functions. Defined with the `lambda` keyword.

Syntax: `lambda arguments: expression`

```
square = lambda x: x ** 2
add = lambda x, y: x + y

# Common use — as argument to higher-order functions
sorted_list = sorted(employees, key=lambda e: e.salary)
filtered = filter(lambda x: x > 18, ages)
doubled = list(map(lambda x: x * 2, numbers))
```

Limitations:

- Only **one expression** — no statements, no multiple lines
- No `if` blocks, loops, or `return` keyword

- Harder to debug — no function name in tracebacks
- Reduced readability for complex logic

Real-World Example: A data analyst sorts a DataFrame of sales records: `df.sort_values(by='revenue', key=lambda col: col.str.replace('$', '').astype(float))`. The lambda strips the dollar sign for proper numeric sorting — a perfect use case: simple, one-time logic that doesn't deserve a named function. But if the logic grows to 5+ steps, a proper `def` function is always preferred.

PART C: Advanced Concepts

Q32. What are generators in Python? How do they differ from lists?

Answer:

Generators are functions that use `yield` instead of `return` — they produce values **one at a time, on demand** (lazily), rather than computing and storing everything upfront.

Feature	List	Generator
Memory	Stores all values in RAM	Generates one value at a time
Creation	<code>[x**2 for x in range(n)]</code>	<code>(x**2 for x in range(n))</code> or <code>yield</code> function
Reusable	✅ Multiple times	❌ Once exhausted, done
Speed (first value)	Slow (compute all first)	Fast (compute on demand)
Use when	All values needed at once	Large/infinite sequences

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

gen = fibonacci()
print([next(gen) for _ in range(10)]) # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Real-World Example: Processing a 50GB log file line by line: `open()` returns a generator — Python reads one line at a time. `for line in open("huge_log.txt")` uses ~1KB of memory regardless of file size. If you used `readlines()` (returns a list), 50GB loads into RAM → system crash. Generators are essential for big data pipelines at companies like Twitter that process billions of log events.

Q33. Explain the difference between shallow copy and deep copy.

Answer:

- **Shallow Copy:** Creates a new object, but **references the same nested objects** as the original. Changes to nested mutable objects affect both copies.
- **Deep Copy:** Creates a completely **independent copy** — all nested objects are also recursively copied. No shared references.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0].append(99)

print(shallow)    # [[1, 2, 99], [3, 4]] – shallow copy affected!
print(deep)       # [[1, 2], [3, 4]] – deep copy unaffected
```

Real-World Example: A game saving system: the current game state includes player inventory (**list** of items). A shallow copy for an "undo" feature would share the inventory list — when the player adds an item, the "undo" state also changes (wrong!). Using **deepcopy** creates a fully independent snapshot — truly independent undo states. Game state managers always use deep copy when saving checkpoints.

Q34. What are context managers? How do you implement them?

Answer:

Context managers define setup and teardown behavior around a block of code using the **with** statement. They guarantee cleanup happens even if an exception occurs.

Implementation approaches:

1. **Class-based:** Implement **__enter__** (setup) and **__exit__** (cleanup)
2. **@contextmanager decorator:** Use **yield** in a generator function

```
# Class-based
class DatabaseConnection:
    def __enter__(self):
        self.conn = connect_to_db()
        return self.conn

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.conn.close()          # always runs
        return False               # don't suppress exceptions

with DatabaseConnection() as conn:
    conn.execute("SELECT * FROM users")
# conn.close() called automatically – even if query fails
```

```
# Generator-based
from contextlib import contextmanager

@contextmanager
def timer():
    start = time.time()
    yield
    print(f"Elapsed: {time.time() - start:.2f}s")

with timer():
    heavy_computation()
```

Real-World Example: A data pipeline that writes to a CSV file: `with open("output.csv", "w") as f:` — the file is guaranteed to close (flushing all data) even if the writing loop crashes midway. Without `with`, a crash might leave the file open with partially written, unclosed data — corrupting it. Python's `with` statement is the standard for **resource management** (files, locks, DB connections, network sockets).

Q35. How does exception handling work in Python?

Answer:

Python handles errors using `try-except-else-finally` blocks. Exceptions are objects in a class hierarchy.

```
try:
    result = int(input("Enter number: ")) # might raise ValueError
    x = 10 / result                       # might raise ZeroDivisionError
except ValueError as e:
    print(f"Not a valid number: {e}")
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:                  # catch-all (use sparingly)
    print(f"Unexpected error: {e}")
else:
    print(f"Result: {x}")                # runs only if NO exception occurred
finally:
    print("Always runs – cleanup here")
```

Custom Exceptions:

```
class InsufficientFundsError(Exception):
    def __init__(self, amount, balance):
        super().__init__(f"Need ₹{amount}, only ₹{balance} available")
```

Real-World Example: A payment API calls an external gateway. `except requests.Timeout` catches network timeouts and retries. `except PaymentDeclinedError` catches card rejections and returns a user-friendly

message. **finally** logs the transaction attempt to an audit trail — regardless of success or failure. Proper exception hierarchy ensures each error type gets the right response, not a generic crash.

Q36. What are list comprehensions? How do they differ from loops?

Answer:

List comprehensions provide a concise, readable way to create lists from existing iterables — often in a single line.

Syntax: `[expression for item in iterable if condition]`

```
# Traditional loop
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x ** 2)

# List comprehension – same result
squares = [x ** 2 for x in range(10) if x % 2 == 0]

# Dictionary comprehension
word_lengths = {word: len(word) for word in ["apple", "banana", "cherry"]}

# Set comprehension
unique_domains = {email.split("@")[1] for email in email_list}
```

Differences:

- Comprehensions are **faster** (optimized bytecode)
- More **readable** for simple transformations
- For complex logic (3+ conditions, side effects) → use regular loops

Real-World Example: A data cleaning pipeline at a retail company: `cleaned_prices = [float(p.replace("$","").strip()) for p in raw_prices if p != "N/A"]` — strips currency symbols and filters missing values in one readable line. What would take a 5-line loop becomes a self-documenting expression that any Python developer immediately understands.

PART D: Libraries & Applications

Q37. What Python libraries are essential for data science?

Answer:

Library	Purpose
NumPy	Numerical computing, n-dimensional arrays, linear algebra

Library	Purpose
Pandas	Data manipulation, DataFrames, CSV/Excel/SQL I/O
Matplotlib / Seaborn	Data visualization, plotting
Scikit-learn	Classical ML algorithms, preprocessing, model evaluation
TensorFlow / PyTorch	Deep learning frameworks
SciPy	Scientific computing, statistics, optimization
Statsmodels	Statistical tests, regression analysis
Jupyter	Interactive notebook environment
NLTK / spaCy	Natural Language Processing
OpenCV	Computer vision

Real-World Example: A data scientist at a healthcare startup builds a patient readmission predictor:

- **Pandas:** Load and clean hospital records from CSV
 - **Seaborn:** Visualize readmission rates by age group
 - **NumPy:** Handle numerical operations efficiently
 - **Scikit-learn:** Train and evaluate XGBoost classifier
 - **Matplotlib:** Plot ROC curves for the medical team
- All five libraries work together seamlessly in a Jupyter Notebook — a typical data science workflow.

Q38. How would you use Python for machine learning?

Answer:

Python is the primary language for ML due to its rich ecosystem. A typical ML workflow in Python:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# 1. Load data
df = pd.read_csv("data.csv")

# 2. Preprocess
X = df.drop("target", axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 3. Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# 4. Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 5. Evaluate
print(classification_report(y_test, model.predict(X_test)))
```

Real-World Example: A fintech startup uses Python to predict credit risk. Pandas loads loan applications from a PostgreSQL database. Scikit-learn preprocesses (imputes missing income, encodes employment type, scales amounts). A GradientBoostingClassifier is trained and achieves 91% AUC. The model is saved with `joblib.dump()` and deployed via a Flask API — all in Python, end to end.

Q39. What is NumPy? Why is it faster than pure Python?

Answer:

NumPy is Python's core library for numerical computing. Its central object is the **ndarray** (n-dimensional array) — a fixed-type, contiguous block of memory.

Why NumPy is faster than pure Python:

Reason	Explanation
Contiguous memory	Elements stored adjacently in RAM → CPU cache friendly
Fixed data type	No type-checking overhead per element
C/Fortran under the hood	Core operations implemented in compiled C — not Python
Vectorization	Operations apply to entire array at once — no Python for-loop
BLAS/LAPACK	Optimized linear algebra libraries for matrix operations

```
import numpy as np
import time

n = 10_000_000
py_list = list(range(n))
np_array = np.arange(n)

# Python loop: ~1.5 seconds
# NumPy vectorized: ~0.01 seconds – 150x faster
result = np_array * 2 + 5 # applies to all 10M elements instantly
```

Real-World Example: A quantitative analyst at a hedge fund computes the dot product of a 10,000 × 10,000 portfolio covariance matrix daily. Pure Python nested loops: ~4 hours. NumPy with BLAS: ~0.3 seconds. Without NumPy, modern quant finance, scientific computing, and deep learning would be computationally infeasible in Python.

Q40. Explain pandas DataFrames and their advantages.

Answer:

A **DataFrame** is Pandas' 2D labeled data structure — rows have index labels, columns have names. Think: a Python spreadsheet/SQL table with superpowers.

Advantages:

Feature	Benefit
Labeled axes	Access data by name, not just position
Mixed data types	Each column can be int, float, string, datetime
Built-in I/O	Read/write CSV, Excel, SQL, JSON, Parquet
Vectorized operations	Fast operations on entire columns
Missing data handling	NaN support with <code>fillna()</code> , <code>dropna()</code>
GroupBy	SQL-like aggregations
Merge/Join	SQL-style table joining

```
df = pd.read_csv("sales.csv")
df['revenue'] = df['price'] * df['quantity']           # new column
top_products = df.groupby('product')['revenue'].sum()  # GroupBy
                .sort_values(ascending=False).head(10) # Top 10
monthly = df[df['date'].dt.month == 6]                 # filter June
```

Real-World Example: A retail analyst at Walmart loads a 5-million-row sales dataset into a DataFrame. With 3 lines: filter last quarter, group by store region, compute average basket size per region. What would take a junior analyst hours in Excel takes seconds in Pandas — and runs the same way on next quarter's data without any changes. Pandas is why Python replaced Excel/R in most corporate analytics teams.



SECTION 3: DATA STRUCTURES



PART A: Fundamentals

Q41. What are data structures? Why are they fundamental to CS?

Answer:

Data structures are organized ways to **store, manage, and access data** efficiently. They define not just how data is stored, but the operations available on it (insert, delete, search, traverse).

Why fundamental:

- Every algorithm needs data organized in some form
- The right data structure = dramatic performance gains
- Wrong choice = correct but impossibly slow program
- Real-world systems are designed around data structure choices

Major categories:

- **Linear:** Arrays, Linked Lists, Stacks, Queues
- **Non-linear:** Trees, Graphs
- **Hash-based:** Hash Tables
- **Specialized:** Heaps, Tries, Segment Trees

Real-World Example: Google Maps computes the shortest driving route between two cities. The road network is modeled as a **weighted graph**. The routing algorithm uses a **priority queue (min-heap)** for efficiency. Without these data structures, finding a route between Delhi and Mumbai might take minutes instead of milliseconds. Choosing the right data structure literally defines the user experience.

Q42. What is the difference between abstract data types and data structures?

Answer:

	Abstract Data Type (ADT)	Data Structure
Definition	Mathematical model defining WHAT operations are possible and their semantics	Concrete implementation defining HOW data is organized in memory
Focus	Behavior and interface	Implementation details
Example	Stack ADT: push, pop, peek, isEmpty	Stack implemented using an array or linked list
Analogy	Interface/Contract	Class/Implementation

ADT defines the "what"; Data Structure defines the "how."

The same ADT can have multiple data structure implementations with different performance characteristics.

Real-World Example: A **Queue ADT** says: you can **enqueue** (add to back), **dequeue** (remove from front), and **peek** (view front). This is the contract. The actual data structure could be:

- A **circular array** — fixed size, cache-friendly
 - A **linked list** — dynamic size, more memory An airline's boarding system uses the Queue ADT — engineers chose the linked list implementation because they don't know how many passengers will board. The gate agent's software doesn't care about the implementation — just that first-in, first-out works correctly.
-

Q43. Explain time complexity and space complexity.

Answer:

- **Time Complexity:** Measures how the **runtime** of an algorithm grows as the input size (n) increases. Expressed as $T(n)$.
- **Space Complexity:** Measures how much **extra memory** an algorithm uses as input grows. Expressed as $S(n)$.

Why we measure growth rate, not exact time:

- Exact time depends on hardware, language, OS
- Growth rate is universal and comparable

Types of analysis:

- **Best case (Ω):** Minimum operations (lucky scenario)
- **Average case (Θ):** Expected operations (typical scenario)
- **Worst case (O):** Maximum operations (most important for guarantees)

Real-World Example: A hospital search system with 1 million patient records:

- **Linear search ($O(n)$):** 1 million comparisons worst case $\rightarrow$ 0.1 seconds
- **Binary search ($O(\log n)$):** 20 comparisons worst case $\rightarrow$ 0.000002 seconds

Both are "correct" — binary search just needs a sorted list. The $50,000\times$ speedup difference makes binary search essential for real-time medical lookup systems.

Q44. What is Big O notation? Compare common complexities.

Answer:

Big O notation describes the **upper bound** (worst case) of an algorithm's time or space complexity as input n grows toward infinity. It ignores constants and lower-order terms.

Common complexities (slowest to fastest):

Big O	Name	Example
$O(1)$	Constant	Hash table lookup
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort, Heap sort
$O(n^2)$	Quadratic	Bubble sort, nested loops
$O(2^n)$	Exponential	Recursive Fibonacci, brute-force
$O(n!)$	Factorial	Traveling salesman (brute force)

Growth comparison for $n = 1,000$: $O(1)=1$, $O(\log n)\approx 10$, $O(n)=1,000$, $O(n \log n)=10,000$, $O(n^2)=1,000,000$, $O(2^n)$ = astronomical

Real-World Example: An e-commerce site processes 1 million product searches per day. A product lookup using $O(n)$ linear scan on 100,000 products = 100,000 operations per search $\times$ 1M searches = 100 billion operations/day. With $O(1)$ hash table lookup — 1 operation per search $\times$ 1M searches = 1 million operations. Big O analysis is why engineers choose hash tables over lists for lookup-heavy applications.

PART B: Arrays & Strings

Q45. Compare static arrays vs dynamic arrays.

Answer:

Feature	Static Array	Dynamic Array
Size	Fixed at declaration	Grows/shrinks automatically
Memory	Allocated at compile/declaration time	Allocated on heap at runtime
Access	$O(1)$	$O(1)$
Insert at end	$O(1)$ if space,  if full	$O(1)$ amortized
Language	C/C++ arrays	Python list, Java ArrayList, C++ vector
Memory waste	May waste (if oversized)	Managed internally

Real-World Example: A chess program's board is a **static 8×8 array** — the board never changes size (always 64 squares), so a fixed `board[8][8]` is perfect. But a chess game's move history is a **dynamic array** — you don't know how many moves will be played. Python's `list` or Java's `ArrayList` grows automatically as moves are added: 1 move, 50 moves, 200 moves — the array handles it all without the programmer managing memory.

Q46. What are the time complexities of array operations?

Answer:

Operation	Static/Dynamic Array	Notes
Access by index	$O(1)$	Direct memory calculation
Search (unsorted)	$O(n)$	Must check each element
Search (sorted)	$O(\log n)$	Binary search
Insert at end	$O(1)$ amortized	$O(n)$ if resize needed
Insert at beginning/middle	$O(n)$	Must shift elements
Delete at end	$O(1)$	
Delete at beginning/middle	$O(n)$	Must shift elements

Why O(1) access? Given base address and index: $\text{address} = \text{base} + \text{index} \times \text{element_size}$ — direct calculation, no traversal needed.

Real-World Example: A music streaming app stores a playlist of 10,000 songs in an array. Jumping to song #5000 directly is O(1) — just compute the memory address. But inserting a new song at position 2 means shifting 9,998 songs one spot — O(n). This is why Spotify likely uses a more sophisticated structure (linked list or indexed database) for large playlists with frequent mid-insertions.

Q47. How do dynamic arrays resize? What is amortized analysis?

Answer:

When a dynamic array is full and a new element is added:

1. Allocate a new, larger array (typically **2 × the current size**)
2. Copy all existing elements to the new array — O(n)
3. Add the new element
4. Free the old array

Why doubling? Doubling strategy ensures resizing happens rarely — after n insertions, only O(log n) resizes occur.

Amortized Analysis: Even though one insertion might cost O(n) (during resize), when you average the cost over all n insertions, each insertion costs **O(1) amortized**. Think: you pay more rarely but the total cost per operation averages out.

Proof sketch: With doubling, total copy work = $1 + 2 + 4 + \dots + n < 2n = O(n)$ for n insertions → O(1) per insertion.

Real-World Example: Java's `ArrayList` starts with capacity 10. When the 11th element is added: creates array of size 20, copies 10 elements (O(10)), adds element 11. This O(n) cost is a "one-time investment" — the next 9 insertions are O(1). Amortized over 20 insertions total, cost = (10 copies + 20 adds) / 20 = 1.5 operations/insertion → effectively O(1). This is why appending to Python lists feels instantaneous despite the occasional hidden resize.

Q48. What string matching algorithms exist?

Answer:

Algorithm	Time Complexity	Key Idea
Naive/Brute Force	$O(n \times m)$	Slide pattern, check each position
KMP (Knuth-Morris-Pratt)	$O(n + m)$	Precompute failure function, never backtrack
Boyer-Moore	$O(n/m)$ best	Skip characters using bad character heuristic
Rabin-Karp	$O(n + m)$ average	Rolling hash to compare substrings
Aho-Corasick	$O(n + m + k)$	Multiple patterns simultaneously using trie

Where n = text length, m = pattern length, k = number of matches.

Real-World Example: A content moderation system at a social media platform needs to detect 10,000 banned phrases in user posts. Naive approach: $O(\text{post_length} \times 10,000 \times \text{average_phrase_length})$ — impossibly slow at scale. **Aho-Corasick** builds a trie of all 10,000 phrases, then scans each post exactly once — $O(\text{post_length} + \text{matches})$. Facebook/Instagram uses Aho-Corasick-based algorithms to moderate billions of posts daily in real time.



PART C: Linked Lists

Q49. Compare singly, doubly, and circular linked lists.

Answer:

Feature	Singly Linked	Doubly Linked	Circular Linked
Node pointers	next only	next + prev	Last node → first node
Traversal	Forward only	Both directions	Can loop continuously
Memory per node	Less (1 pointer)	More (2 pointers)	Depends on type
Delete given node	$O(n)$ (need previous)	$O(1)$ (has prev pointer)	$O(n)$ or $O(1)$
Use cases	Stacks, simple lists	Browser history, undo/redo	Round-robin scheduling, music playlist loop

```

Singly:  [A]→[B]→[C]→null
Doubly:  null←[A]↔[B]↔[C]→null
Circular: [A]→[B]→[C]→[A] (loops back)

```

Real-World Example:

- **Singly:** Undo operation in a simple text editor (only need to go backwards one step)
- **Doubly:** Browser's back/forward history — Chrome stores pages in a doubly linked list; clicking back traverses **prev**, clicking forward traverses **next**
- **Circular:** An operating system's CPU scheduler gives each process a time slice in a **round-robin** fashion using a circular linked list — after the last process gets its turn, it wraps back to the first

Q50. What are the advantages of linked lists over arrays?

Answer:

Advantage	Explanation
Dynamic size	Grows/shrinks at runtime without declaring size upfront
O(1) insert/delete	At known position — just change pointers, no shifting
No memory waste	Allocates exactly what's needed (vs. array over-allocation)
No contiguous memory needed	Nodes can be scattered in RAM

Disadvantages:

- O(n) access (no random access — must traverse from head)
- Extra memory per node (pointer storage)
- Poor cache locality (scattered in RAM, cache misses)

Real-World Example: A hospital's patient queue: patients arrive and leave unpredictably. With an array, inserting a high-priority patient in the middle requires shifting all subsequent patients — O(n). With a **linked list**, inserting a new priority patient between nodes 3 and 4 just changes two pointers — O(1). The linked list's flexibility makes it ideal for dynamic priority queuing in emergency rooms.

Q51. How do you detect a cycle in a linked list?

Answer:

Floyd's Cycle Detection Algorithm (Tortoise and Hare):

Use two pointers:

- **slow** moves **1 step** at a time
- **fast** moves **2 steps** at a time

If there's a cycle, **fast** will eventually catch up to **slow** (they'll meet inside the cycle). If there's no cycle, **fast** reaches **null**.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True # cycle detected
    return False

# Time: O(n), Space: O(1) – no extra data structure needed
```

Alternatives: Use a HashSet to store visited nodes — O(n) space.

Real-World Example: A network packet routing system maintains a linked list of routers that a packet will hop through. If a misconfigured router table creates a loop (router A → B → C → A), packets loop forever,

consuming bandwidth infinitely — a real "routing loop" problem. Floyd's algorithm detects such cycles in network path validation, helping network diagnostic tools like `traceroute` detect infinite routing loops.

Q52. How do you reverse a linked list?

Answer:

Iterative approach (O(n) time, O(1) space):

```
def reverse_linked_list(head):
    prev = None
    current = head

    while current:
        next_node = current.next # save next
        current.next = prev      # reverse the link
        prev = current           # move prev forward
        current = next_node      # move current forward

    return prev # new head (was the old tail)
```

Recursive approach:

```
def reverse_recursive(head):
    if not head or not head.next:
        return head
    new_head = reverse_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head
```

Real-World Example: A browser's "back" button uses a history stack. If we represent history as a linked list and need to display it in reverse order (most recent first), we reverse it. Git also reverses commit chains when displaying `git log --reverse`. The iterative approach is preferred in production — no risk of stack overflow for very long histories (recursive approach fails if there are 10,000+ commits).

PART D: Stacks & Queues

Q53. Explain stacks and queues as ADTs.

Answer:

Stack — LIFO (Last In, First Out):

- `push(item)` — add to top
- `pop()` — remove from top

- `peek()` — view top without removing
- `isEmpty()` — check if empty

Queue — FIFO (First In, First Out):

- `enqueue(item)` — add to back
- `dequeue()` — remove from front
- `peek()` — view front
- `isEmpty()` — check if empty

Feature	Stack	Queue
Order	LIFO	FIFO
Add to	Top	Back (rear)
Remove from	Top	Front
Analogy	Stack of plates	Line at ticket counter

Real-World Example:

- **Stack:** When you press Ctrl+Z (Undo) in Microsoft Word, the last action is undone first (LIFO). Each action is pushed onto an "undo stack." Undo pops the top action and reverses it.
- **Queue:** A printer spool — print jobs are processed in the order they were submitted (FIFO). The document you submitted first gets printed first, regardless of file size.

Q54. What are the applications of stacks?

Answer:

Application	How Stack is Used
Function call management	Call stack — tracks active function calls and local variables
Undo/Redo	Each action pushed; undo pops; redo uses a second stack
Expression evaluation	Evaluate postfix expressions (RPN calculators)
Syntax parsing	Check balanced parentheses: <code>{[()]}</code>
Browser history	Back button uses a stack of visited pages
DFS (Graph/Tree traversal)	Explicit stack or implicit recursion stack
Backtracking algorithms	Maze solving, N-Queens, Sudoku

Real-World Example: Every time you call a function in any program, the CPU pushes a **stack frame** (containing local variables, parameters, return address) onto the **call stack**. When the function returns, the frame is popped. This is why infinite recursion causes a **StackOverflowError** — the call stack (limited memory) keeps growing with no pops. JVM sets a default thread stack size of 512KB, fitting roughly 5,000 recursive calls before overflow.

Q55. Compare simple queue, circular queue, and priority queue.

Answer:

Feature	Simple Queue	Circular Queue	Priority Queue
Order	FIFO strictly	FIFO (wraps around)	By priority value
Memory	Linear	Fixed, reuses space	Dynamic
Problem solved	Basic ordering	Avoids "false full"	Urgent-first processing
Implementation	Array or linked list	Array with modular indexing	Heap (binary heap)
Complexity (enqueue/dequeue)	O(1)	O(1)	O(log n)

Circular Queue: When front/rear pointers reach the end of a fixed array, they wrap to position 0 — reusing dequeued space. Eliminates the "false full" problem of simple array queue.

Real-World Example:

- **Simple Queue:** Call center — customers wait in order they called
- **Circular Queue:** CPU scheduling in OS — processes cycle through time slices using a circular buffer; the CPU's hardware interrupt queue is circular
- **Priority Queue:** Hospital ER triage — patients aren't served in arrival order; a patient with a heart attack (priority 1) jumps ahead of a patient with a minor cut (priority 5). `heapq` in Python implements this

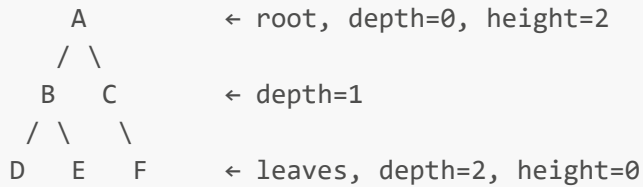
PART E: Trees

Q56. Define tree terminology: root, leaf, height, depth.

Answer:

Term	Definition
Root	The topmost node — has no parent
Leaf	A node with no children
Internal node	A node with at least one child
Parent	A node directly above another
Child	A node directly below another
Sibling	Nodes sharing the same parent
Height of node	Length of longest path from node to a leaf
Height of tree	Height of root node

Term	Definition
Depth of node	Length of path from root to that node
Level	Set of nodes at the same depth
Degree	Number of children a node has



Height of tree = 2, Height of B = 1, Depth of C = 1.

Real-World Example: A company's organizational chart is a tree:

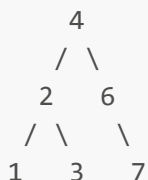
- **Root:** CEO (depth=0)
- **Leaves:** Individual contributors with no direct reports
- **Height:** Number of management layers
- **Depth of a VP:** Number of positions between them and the CEO

HR systems use tree traversals to compute total team size under any manager in $O(\text{subtree size})$ time.

Q57. Explain tree traversals: preorder, inorder, postorder, level-order.

Answer:

Traversal	Order	Use Case
Preorder (Root, Left, Right)	Visit root → left subtree → right subtree	Copy a tree, serialize a tree
Inorder (Left, Root, Right)	Visit left subtree → root → right subtree	Get BST elements in sorted order
Postorder (Left, Right, Root)	Visit left → right → root	Delete a tree, compute directory sizes
Level-order (BFS)	Visit level by level using a queue	Shortest path in unweighted trees, print level by level



Preorder: 4 2 1 3 6 7

```
Inorder:      1 2 3 4 6 7  ← sorted!
Postorder:    1 3 2 7 6 4
Level-order:  4 2 6 1 3 7
```

Real-World Example: A **file system** uses postorder traversal to calculate directory sizes: a folder's size = sum of all files inside it. You must compute sizes of all subdirectories **before** computing the parent's total. Unix **du -sh** (disk usage) effectively does a postorder traversal of the directory tree. Inorder traversal of a BST-based database index returns records in sorted order — exactly how SQL **ORDER BY** works with tree indexes.

Q58. What is a Binary Search Tree? What properties define it?

Answer:

A **Binary Search Tree (BST)** is a binary tree where for every node:

1. All keys in the **left subtree** are **less than** the node's key
2. All keys in the **right subtree** are **greater than** the node's key
3. Both left and right subtrees are also valid BSTs
4. No duplicate keys (typically)

Operations on BST:

- **Search:** Compare, go left if smaller, right if larger — $O(h)$
- **Insert:** Follow search path, insert at null position — $O(h)$
- **Delete:** 3 cases: leaf, one child, two children — $O(h)$
- **Inorder traversal:** Returns all keys in sorted order — $O(n)$

Where h = height of tree.

Real-World Example: A dictionary app's word lookup uses a BST: searching for "mango" — compare with root "parrot": $m < p$, go left. Compare with "lemon": $m > l$, go right. Compare with "mango" — found! In a balanced BST of 1 million words, this takes at most 20 comparisons ($\log_2 1,000,000 \approx 20$). A linear scan of the same dictionary would take up to 1 million comparisons.

Q59. Analyze BST operations in best, average, and worst cases.

Answer:

Operation	Best Case	Average Case	Worst Case
Search	$O(1)$ — root	$O(\log n)$ — balanced	$O(n)$ — skewed
Insert	$O(1)$	$O(\log n)$	$O(n)$
Delete	$O(1)$	$O(\log n)$	$O(n)$
Min/Max	$O(1)$ if stored	$O(\log n)$	$O(n)$
Inorder	$O(n)$	$O(n)$	$O(n)$

Worst case occurs when: Elements are inserted in sorted (or reverse sorted) order → BST degenerates into a **linked list**. Example: inserting 1, 2, 3, 4, 5 → straight line of right children, height = n.

Why balanced trees matter: AVL/Red-Black trees guarantee $O(\log n)$ always by rebalancing after insertions/deletions.

Real-World Example: A naive BST for storing user IDs that are assigned sequentially (1, 2, 3, 4...) degenerates into a right-leaning linked list — searching for user 1,000,000 requires 1 million comparisons. A production database uses a **B-Tree** (balanced variant) which guarantees $O(\log n)$ even for sequential insertions — that's why PostgreSQL and MySQL use B-Trees for indexes, not simple BSTs.

Q60. What are balanced trees? Why do we need them?

Answer:

A **balanced tree** maintains its height as $O(\log n)$ — ensuring efficient operations regardless of insertion order. Balance is maintained by **rotations** or structural adjustments after each modification.

Problem without balance: A BST's performance degrades to $O(n)$ when it becomes unbalanced (skewed). Balanced trees prevent this.

Common balanced tree types:

- **AVL Tree:** Strictly balanced — height difference between subtrees ≤ 1 at every node
- **Red-Black Tree:** Approximately balanced — height $\leq 2 \log(n+1)$ — used in Java's TreeMap, C++ STL
- **B-Tree/B+ Tree:** Multi-way balanced tree — used in databases and file systems
- **2-3 Tree:** Educational predecessor to Red-Black trees

Real-World Example: Linux kernel's process scheduler uses a **Red-Black Tree** to manage runnable processes. Processes are ordered by "virtual runtime." The scheduler always picks the leftmost node (minimum virtual runtime) for the next CPU slot. Even with thousands of processes being inserted/deleted/searched every millisecond, Red-Black tree guarantees $O(\log n)$ — critical for real-time scheduling fairness.

Q61. Compare AVL trees and Red-Black trees.

Answer:

Feature	AVL Tree	Red-Black Tree
Balance condition		$\text{height}(L) - \text{height}(R)$
Height guarantee	$\leq 1.44 \log(n)$	$\leq 2 \log(n+1)$
Rotations (insert)	At most 2	At most 2
Rotations (delete)	$O(\log n)$	At most 3
Search performance	Faster (more balanced)	Slightly slower
Insert/Delete	Slower (more rebalancing)	Faster
Memory	1 extra int (balance factor)	1 extra bit (color)

Feature	AVL Tree	Red-Black Tree
Best for	Read-heavy workloads	Write-heavy workloads
Used in	Databases requiring fast lookup	Java TreeMap, C++ std::map, Linux scheduler

Real-World Example:

- **AVL Tree:** A **DNS lookup system** — queried millions of times per second, rarely updated. AVL's stricter balance means faster lookups — perfect for read-heavy systems.
- **Red-Black Tree:** A **stock trading system** with constant order insertions and cancellations. Red-Black's faster insert/delete (fewer rotations) handles the high write throughput better. Java's **TreeMap** (used to maintain sorted order books in trading systems) uses a Red-Black tree internally.



PART F: Heaps

Q62. What is a heap? Explain min-heap vs max-heap.

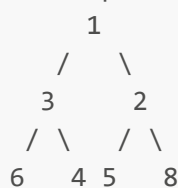
Answer:

A **heap** is a complete binary tree (all levels fully filled except possibly the last, filled left to right) that satisfies the **heap property**:

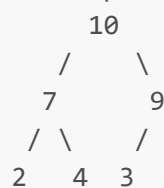
- **Min-Heap:** Every parent node is $\leq$ its children $\rightarrow$ root is the **minimum** element
- **Max-Heap:** Every parent node is $\geq$ its children $\rightarrow$ root is the **maximum** element

Key property: The root always gives you the min (or max) in $O(1)$ — extremely useful for priority queues.

Min-Heap:



Max-Heap:



Real-World Example: A hospital emergency system uses a **min-heap** where lower numbers = higher priority. Patient with severity 1 is at the root — always treated first. When a severity-1 patient arrives, they're inserted and bubble up to the top. When treated, the root is removed and the heap restructures in $O(\log n)$ — guaranteeing the most critical patient is always seen next.

Q63. How is a heap implemented using arrays?

Answer:

A complete binary tree can be **efficiently stored in an array** using index arithmetic — no explicit node-pointer storage needed.

Index mapping (1-indexed):

- Root → index 1
- Left child of node i → index $2i$
- Right child of node i → index $2i + 1$
- Parent of node i → index $i // 2$

0-indexed version (Python's heapq):

- Left child of i → $2i + 1$
- Right child of i → $2i + 2$
- Parent of i → $(i - 1) // 2$

Heap: [1, 3, 2, 6, 4, 5, 8] (array)

Represents:

1

/ \

3 2

/ \ / \

6 4 5 8

(index 0)

(index 1, 2)

(index 3, 4, 5, 6)

Real-World Example: Python's `heapq` module implements a min-heap using a plain list. Task schedulers in Python use `heapq.heappush(tasks, (priority, task))` and `heapq.heappop(tasks)` — no separate heap class or node objects needed. The array representation makes heaps extremely **cache-friendly** (contiguous memory) and memory-efficient compared to a pointer-based tree — critical for high-performance systems.

Q64. What are the time complexities of heap operations?

Answer:

Operation	Time Complexity	Explanation
Get min/max (peek)	$O(1)$	Root always holds it
Insert (heappush)	$O(\log n)$	Insert at end, bubble up
Extract min/max (heappop)	$O(\log n)$	Remove root, replace with last, bubble down
Build heap from array (heapify)	$O(n)$	Better than $O(n \log n)$ — Floyd's algorithm
Decrease key	$O(\log n)$	Update value, bubble up
Delete arbitrary element	$O(\log n)$	After decrease-key to $-\infty$, extract
Search	$O(n)$	No ordering between siblings

Heapify is $O(n)$ — not $O(n \log n)$: Most insertions happen at lower levels where bubbling up is short. Mathematical proof: sum of work at all levels = $O(n)$.

Real-World Example: Dijkstra's shortest path algorithm uses a min-heap as its priority queue. For a city road network with $n=10,000$ intersections and $m=50,000$ roads: with a simple array priority queue, it's $O(n^2) = 100$ million operations. With a binary heap: $O((n + m) \log n) \approx 600,000$ operations — 166× faster. Google Maps uses similar heap-optimized shortest path algorithms to compute routes in real time.

PART G: Hashing

Q65. Explain hashing and hash functions.

Answer:

Hashing is the process of converting input data (key) into a fixed-size value (hash code / digest) using a **hash function**. The hash code is used to determine where to store/retrieve data in a hash table.

Properties of a good hash function:

1. **Deterministic:** Same input always gives same hash
2. **Fast to compute:** $O(1)$ ideally
3. **Uniform distribution:** Hashes spread evenly across buckets (minimize collisions)
4. **Avalanche effect:** Small input change → completely different hash

Common hash functions:

- Division method: $h(k) = k \bmod m$
- Multiplication method: $h(k) = \text{floor}(m \times (k \times A \bmod 1))$
- Cryptographic: MD5, SHA-256 (for security, not lookup speed)

Real-World Example: Git uses **SHA-1** hashing (now SHA-256) to identify every commit, file, and tree object. When you commit code, Git hashes the entire file content → produces a 40-character hash like `3a7f2b...`. Two files with even one character difference produce completely different hashes — making it impossible to tamper with history undetected. This is why git history is immutable and auditable.

Q66. What are hash collisions? Compare chaining vs open addressing.

Answer:

A **collision** occurs when two different keys produce the same hash (map to the same bucket). Collisions are inevitable (birthday paradox). Two main resolution strategies:

Chaining:

- Each bucket holds a linked list of all entries with that hash
- Insert: append to list — $O(1)$
- Search: traverse list — $O(1)$ average, $O(n)$ worst (all collide)
- Load factor can exceed 1; extra memory for pointers

Open Addressing:

- All entries stored inside the array itself

- On collision, probe for the next empty slot
 - **Linear probing:** Check next slot, next+1, ...
 - **Quadratic probing:** Check +1², +2², +3²...
 - **Double hashing:** Use second hash for step size
- Load factor must stay < 1; better cache performance (no pointers)

	Chaining	Open Addressing
Extra memory	Linked list nodes	No extra (in-array)
Cache	Poor (linked list)	Good (array locality)
Load factor	Can exceed 1	Must stay < 1
Used in	Java HashMap	Python dict, CPython, linear probing

Real-World Example: Python's dictionary uses **open addressing** with a custom probing sequence (not simple linear) — optimized for CPU cache performance since all data stays in a contiguous array. Java's HashMap uses **chaining** (linked list → Red-Black Tree for long chains). Python's approach is faster for small tables (no pointer-following); Java's is safer for high collision scenarios.

Q67. Compare hash tables with balanced BSTs.

Answer:

Feature	Hash Table	Balanced BST (AVL/RB)
Search	O(1) average	O(log n)
Insert	O(1) average	O(log n)
Delete	O(1) average	O(log n)
Min/Max	O(n)	O(log n)
Range query	O(n)	O(log n + k)
Sorted order	✗ Not maintained	✓ Inorder traversal
Ordering	None	Sorted by key
Memory	May waste (load factor)	Per-node overhead
Worst case	O(n) with bad hash	O(log n) guaranteed

Choose Hash Table when: Fast average-case lookup, no ordering needed. **Choose BST when:** Need sorted data, range queries, min/max, or guaranteed worst-case performance.

Real-World Example: A social network needs two operations:

- "Is @username taken?" → **Hash Table** — O(1) lookup, just need yes/no
- "Show all users whose names start with 'A'" → **BST/Trie** — range queries, needs ordering

Twitter uses both: a hash map for $O(1)$ username uniqueness checks and a B-Tree index (on the database) for sorted range queries on usernames/emails.

PART H: Graphs

Q68. Define graphs: vertices, edges, directed vs undirected.

Answer:

A **graph** $G = (V, E)$ consists of:

- **Vertices (V):** Nodes — entities in the graph
- **Edges (E):** Connections between vertices

Types:

Type	Description	Example
Undirected	Edges have no direction — $(A,B) = (B,A)$	Facebook friendship (mutual)
Directed (Digraph)	Edges have direction — $(A \rightarrow B) \neq (B \rightarrow A)$	Twitter follow (one-way)
Weighted	Edges have numeric values (distances, costs)	Road map with distances
Unweighted	All edges equal	Social network connections
Cyclic	Contains at least one cycle	Most social networks
Acyclic	No cycles	DAG — dependency trees
Connected	Path exists between every pair	Single-component graph

Real-World Example: LinkedIn's professional network is an **undirected weighted graph**: people are vertices, connections are edges. The "degrees of separation" feature computes the shortest path between two users. The "People You May Know" feature analyzes common neighbors. LinkedIn's entire recommendation system is built on graph algorithms processing millions of vertices and billions of edges.

Q69. Compare adjacency matrix vs adjacency list representations.

Answer:

Feature	Adjacency Matrix	Adjacency List
Storage	$O(V^2)$	$O(V + E)$
Check edge (u,v)	$O(1)$	$O(\text{degree}(u))$
Find all neighbors	$O(V)$	$O(\text{degree}(u))$
Add edge	$O(1)$	$O(1)$
Space (sparse graph)	Wastes memory	Efficient

Feature	Adjacency Matrix	Adjacency List
Space (dense graph)	Efficient	Slightly more overhead
Best for	Dense graphs (many edges)	Sparse graphs (few edges)

Graph: A-B, A-C, B-D

Matrix:	A B C D	List:
A	[0 1 1 0]	A: [B, C]
B	[1 0 0 1]	B: [A, D]
C	[1 0 0 0]	C: [A]
D	[0 1 0 0]	D: [B]

Real-World Example: The internet has ~5 billion users but the average person has ~338 Facebook friends — a **sparse graph** ($E \ll V^2$). Facebook uses an **adjacency list** — storing only existing friendships. An adjacency matrix would require $5B \times 5B = 25 \times 10^{18}$ entries (petabytes of storage for mostly zeros). In contrast, a **chess board attack map** (which squares can a queen attack?) is dense — a matrix is appropriate.

Q70. Explain Breadth-First Search (BFS) and its applications.

Answer:

BFS explores a graph **level by level** — all neighbors of a node before going deeper. Uses a **queue** (FIFO).

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Properties:

- Finds the **shortest path** (fewest edges) in unweighted graphs
- Time: $O(V + E)$, Space: $O(V)$
- Guaranteed to visit all nodes in connected graph

Applications:

- Shortest path in unweighted graphs
- Social network "degrees of separation"
- Web crawlers (crawl pages level by level)
- GPS navigation for nearby points of interest
- Finding connected components

Real-World Example: WhatsApp's "mutual friends" feature uses BFS starting from your profile — level 1 = your direct contacts, level 2 = friends-of-friends. "People you may know" are typically at BFS level 2. Facebook calculates that ~3.5 degrees separate any two users on their platform — computed using BFS on their friend graph. BFS guarantees the minimum degrees of separation, not just any path.

Q71. Explain Depth-First Search (DFS) and its applications.

Answer:

DFS explores a graph by going as **deep as possible** along each branch before backtracking. Uses a **stack** (explicitly or via recursion).

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
    return visited
```

Properties:

- Time: $O(V + E)$, Space: $O(V)$ for visited set + $O(h)$ call stack
- Does NOT guarantee shortest path
- Good for exploring all possibilities

Applications:

- Topological sorting (dependency resolution)
- Detecting cycles
- Solving mazes and puzzles
- Connected components
- Strongly Connected Components (Kosaraju's algorithm)
- Compilation: symbol resolution order

Real-World Example: When `npm install` or `pip install` resolves package dependencies, it uses **DFS-based topological sort**: if package A depends on B, and B depends on C, DFS determines the correct installation order ($C \rightarrow B \rightarrow A$). If there's a **circular dependency** (A needs B, B needs A), DFS detects the cycle and throws an error. Package managers worldwide use DFS for dependency resolution.

Q72. What is Dijkstra's algorithm? When does it fail?

Answer:

Dijkstra's algorithm finds the **shortest path** from a single source to all other vertices in a **weighted graph with non-negative edge weights**.

Algorithm:

1. Initialize all distances as ∞ , source distance = 0
2. Use a min-heap (priority queue) of (distance, vertex)
3. Extract vertex with minimum distance
4. For each neighbor: if $\text{current_dist} + \text{edge_weight} < \text{known_dist} \rightarrow$ update and push to heap
5. Repeat until heap is empty

Time Complexity: $O((V + E) \log V)$ with binary heap

When Dijkstra FAILS:

- **Negative edge weights** $\rightarrow$ may give wrong answers (use **Bellman-Ford** instead)
- **Negative cycles** $\rightarrow$ algorithm loops infinitely

```
import heapq

def dijkstra(graph, source):
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    heap = [(0, source)]

    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]: continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

Real-World Example: Google Maps computes the fastest driving route using a Dijkstra variant. Intersections are vertices, roads are weighted edges (weight = travel time). The algorithm finds the path from your location to the destination with minimum total travel time. It fails if edges had "negative time" (impossible in real roads) — which is why Dijkstra works perfectly for navigation. For financial arbitrage graphs where "negative edges" (profit opportunities) exist, Bellman-Ford is used instead.



SECTION 4: SQL

 PART A: Fundamentals

Q73. What is SQL? What are different SQL dialects?

Answer:

SQL (Structured Query Language) is the standard language for managing and querying data in **relational database management systems (RDBMS)**. It's declarative — you describe *what* data you want, not *how* to retrieve it.

SQL Dialects (database-specific variations):

Dialect	Database	Key Differences
T-SQL	Microsoft SQL Server	TOP n, GETDATE(), CTEs with WITH
PL/SQL	Oracle	ROWNUM, NVL(), procedural extensions
MySQL	MySQL/MariaDB	LIMIT, AUTO_INCREMENT, IFNULL()
PostgreSQL	PostgreSQL	Most standards-compliant, RETURNING, JSON support
SQLite	SQLite	Lightweight, AUTOINCREMENT, limited ALTER TABLE

Real-World Example: A company migrates from MySQL to PostgreSQL. Their MySQL query uses `LIMIT 10` (works in both) but also `GROUP_CONCAT()` (MySQL-specific) — must be rewritten as `STRING_AGG()` in PostgreSQL. SQL dialects are 90% compatible, but the 10% differences cause bugs during migrations — a common enterprise pain point when switching cloud database providers.

Q74. Explain the types of SQL commands: DDL, DML, DCL, TCL.

Answer:

Category	Full Name	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define/modify database structure (schema)
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Manipulate actual data
DCL	Data Control Language	GRANT, REVOKE	Control access/permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

Key distinction:

- DDL changes are **auto-committed** (can't rollback in most databases)
- DML changes are part of **transactions** (can rollback)

Real-World Example: A bank's database administrator (DBA):

1. **DDL:** `CREATE TABLE accounts (...)` — sets up the accounts table structure
 2. **DML:** `INSERT INTO accounts VALUES (...)` — adds customer data; `UPDATE accounts SET balance = ...` — transfers money
 3. **DCL:** `GRANT SELECT ON accounts TO teller_role` — gives bank tellers read-only access
 4. **TCL:** `BEGIN; UPDATE...; UPDATE...; COMMIT;` — ensures a money transfer either fully completes or fully rolls back
-

Q75. What is a relational database? Explain the relational model.

Answer:

A **relational database** stores data in **tables (relations)** — structured as rows (tuples) and columns (attributes). Tables can be related to each other through keys.

Core concepts of the relational model (Codd's model, 1970):

Concept	Meaning
Relation (Table)	Set of tuples with same attributes — no duplicate rows
Tuple (Row)	One data record
Attribute (Column)	A field with a data type and name
Domain	Allowed values for an attribute
Primary Key	Uniquely identifies each row
Foreign Key	References a primary key in another table
Schema	Structure/definition of a table

Real-World Example: Amazon's product database: **Products** table (ProductID, Name, Price) relates to **Orders** table (OrderID, ProductID, Quantity) through **ProductID** as a foreign key. When you view your order history, a JOIN combines both tables using this relationship — retrieving the product name and price from Products for each row in Orders. The relational model's power is expressing complex relationships without data duplication.

Q76. What is a primary key? What properties must it have?

Answer:

A **primary key (PK)** is a column (or set of columns) that **uniquely identifies each row** in a table.

Required Properties:

1. **Unique:** No two rows can have the same primary key value
2. **Not Null:** Primary key can never be NULL
3. **Immutable (ideally):** Should not change over time (changing a PK breaks foreign key relationships)
4. **Minimal:** No unnecessary columns in composite keys

Types:

- **Natural Key:** Meaningful real-world attribute (e.g., email address, Aadhaar number)
- **Surrogate Key:** Artificially created, no real-world meaning (e.g., auto-increment integer ID)
- **Composite Key:** Multiple columns together form the PK (e.g., StudentID + CourseID in Enrollment table)

Real-World Example: An airline uses **FlightNumber + DepartureDate** as a **composite primary key** for the Flights table — neither alone is unique (same flight number flies daily, same date has many flights), but together they uniquely identify a specific flight. For the Passengers table, using **email** as a natural PK is tempting but risky — people change emails. A surrogate **PassengerID** (auto-increment) is more stable and standard practice.

Q77. Explain foreign keys and referential integrity.

Answer:

A **foreign key (FK)** is a column in one table that references the **primary key** of another table, establishing a link between the two tables.

Referential Integrity: A constraint that ensures FK values always refer to existing, valid PK values in the parent table — preventing "orphan records."

ON DELETE options:

- **RESTRICT/NO ACTION** — Block deletion if referenced rows exist
- **CASCADE** — Delete child rows when parent is deleted
- **SET NULL** — Set FK to NULL when parent deleted
- **SET DEFAULT** — Set FK to a default value

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
        ON DELETE RESTRICT  
        ON UPDATE CASCADE  
);
```

Real-World Example: An e-commerce platform: the **Orders** table has **CustomerID** as a FK referencing **Customers**. If a customer account is deleted, **ON DELETE RESTRICT** prevents deletion if the customer has orders — protecting order history. But with **ON DELETE CASCADE**, deleting a customer automatically deletes all their orders. GDPR's "right to be forgotten" requires careful FK handling — typically setting customer data to NULL while keeping order records for financial auditing.

Q78. What is database normalization? Why is it important?

Answer:

Normalization is the process of organizing a relational database to **reduce data redundancy** and **improve data integrity** by following a set of rules called **Normal Forms**.

Problems normalization solves:

- **Insertion anomaly:** Can't add data without adding unrelated data
- **Update anomaly:** Updating one fact requires changing multiple rows
- **Deletion anomaly:** Deleting a record unintentionally loses other information

Why important:

- Eliminates data duplication → saves storage
- Ensures data consistency — one fact in one place
- Makes updates, inserts, deletes safer

Trade-off: Highly normalized databases require more JOINS (slower reads). Data warehouses often **denormalize** intentionally for read performance.

Real-World Example: An unnormalized school database stores: **StudentName**, **Course**, **Teacher**, **TeacherPhone** in one table. Problem: if a teacher's phone changes, you must update hundreds of rows. With normalization: **Students** table, **Courses** table, **Teachers** table — teacher's phone stored once in **Teachers**. One update changes it everywhere. Universities use 3NF for transactional databases; their reporting/analytics systems use denormalized star schemas for fast queries.

PART B: Normalization

Q79. Explain 1NF, 2NF, 3NF, and BCNF with examples.

Answer:

1NF (First Normal Form):

- Each cell contains **atomic (single)** values
- No repeating groups or arrays
- Each row uniquely identifiable

✗ Violates 1NF: **Courses** = "Math, Physics, CS" (multiple values in one cell) ✓ 1NF: Separate row for each course

2NF (Second Normal Form):

- Must be in 1NF
- Every non-key attribute is **fully functionally dependent** on the **entire** primary key
- Eliminates partial dependencies (only matters for composite PKs)

✗ Violates 2NF: In (**StudentID**, **CourseID**) → **StudentName** — StudentName depends only on StudentID (partial dependency) ✓ 2NF: Move StudentName to Students table

3NF (Third Normal Form):

- Must be in 2NF
- No **transitive dependencies** — non-key attributes must not depend on other non-key attributes

✗ Violates 3NF: $StudentID \rightarrow ZipCode \rightarrow City$ — City depends on ZipCode (not directly on StudentID) ✓
 3NF: Move ZipCode/City to separate ZipCodes table

BCNF (Boyce-Codd Normal Form):

- Stronger version of 3NF
- For every functional dependency $X \rightarrow Y$, X must be a **superkey**
- Handles edge cases 3NF misses (rare, involves overlapping candidate keys)

Real-World Example: A university's enrollment system violates 3NF: `EnrollmentTable(StudentID, CourseID, InstructorID, InstructorOffice)`. InstructorOffice depends on InstructorID (transitive dependency, not on the PK directly). Fix: create `Instructors(InstructorID, InstructorOffice)` table. Now updating an instructor's office is a single-row change — no anomaly. Hospital management systems are carefully normalized to 3NF to prevent medical record inconsistencies.

Q80. What are functional dependencies?

Answer:

A **functional dependency (FD)** $X \rightarrow Y$ means: knowing the value of attribute X **uniquely determines** the value of attribute Y. X "functionally determines" Y.

Types:

- **Trivial FD:** Y is a subset of X (always holds) — $\{A, B\} \rightarrow A$
- **Non-trivial FD:** Y is not a subset of X — $StudentID \rightarrow StudentName$
- **Full FD:** Y depends on the **entire** X (important for 2NF)
- **Partial FD:** Y depends on part of X — violation of 2NF
- **Transitive FD:** $X \rightarrow Z$ through Y ($X \rightarrow Y \rightarrow Z$) — violation of 3NF

Armstrong's Axioms (rules to infer new FDs):

- **Reflexivity:** If $Y \subseteq X$, then $X \rightarrow Y$
- **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$
- **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Real-World Example: In a company payroll database:

- $EmployeeID \rightarrow Name$ (one employee → one name) ✓
- $EmployeeID \rightarrow DepartmentID$ ✓
- $DepartmentID \rightarrow DepartmentName$ ✓ (transitive: $EmployeeID \rightarrow DeptID \rightarrow DeptName$ — 3NF violation!)
- $Name \rightarrow EmployeeID$ ✗ (two employees can have the same name)

Understanding FDs is what allows a DBA to normalize the payroll system correctly, preventing the salary update of one employee from accidentally affecting another's records.

PART C: Queries — Basic

Q81. Explain the structure and execution order of a SELECT statement.

Answer:

Writing order:

```
SELECT columns
FROM table
JOIN other_table ON condition
WHERE filter_condition
GROUP BY columns
HAVING group_filter
ORDER BY columns
LIMIT n
```

Execution order (logical processing):

- | | |
|-----------------|--|
| 1. FROM / JOIN | → Identify source tables, create working set |
| 2. WHERE | → Filter individual rows |
| 3. GROUP BY | → Group filtered rows |
| 4. HAVING | → Filter groups |
| 5. SELECT | → Choose/compute columns |
| 6. DISTINCT | → Remove duplicates |
| 7. ORDER BY | → Sort results |
| 8. LIMIT/OFFSET | → Restrict output rows |

Why this matters: You can't use a column alias from **SELECT** in **WHERE** — **WHERE** executes before **SELECT** processes aliases. Use a subquery or CTE instead.

Real-World Example: An analyst writes: **SELECT** department, **AVG**(salary) as avg_sal **FROM** employees **WHERE** active=1 **GROUP BY** department **HAVING** **AVG**(salary) > 50000 **ORDER BY** avg_sal **DESC**. Execution: **FROM** gets all rows → **WHERE** keeps only active employees → **GROUP BY** groups by department → **HAVING** filters to departments with avg salary > 50K → **SELECT** computes the average → **ORDER BY** sorts results. The analyst sees a ranked list of well-paying active departments.

Q82. What is the difference between **WHERE** and **HAVING** clauses?

Answer:

Feature	WHERE	HAVING
Purpose	Filter individual rows	Filter groups
Execution	Before GROUP BY	After GROUP BY

Feature	WHERE	HAVING
Use with aggregates	✗ Cannot use aggregate functions	✓ Can use aggregate functions
Operates on	Raw rows	Grouped results

```
-- WHERE filters rows BEFORE grouping
SELECT department, AVG(salary)
FROM employees
WHERE age > 25 -- filters individual employees
GROUP BY department

-- HAVING filters groups AFTER grouping
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000 -- filters department groups
```

Real-World Example: An HR dashboard: "Show departments where the average salary of employees hired after 2020 exceeds ₹80,000." The query needs **both**: `WHERE hire_date > '2020-01-01'` to first exclude old employees from the average calculation, then `HAVING AVG(salary) > 80000` to filter departments by that average. Using WHERE to filter groups or HAVING to filter rows are common SQL mistakes that produce wrong results.

Q83. Explain aggregate functions: COUNT, SUM, AVG, MIN, MAX.

Answer:

Aggregate functions **compute a single result from multiple rows** — typically used with `GROUP BY`.

Function	Purpose	NULL handling
<code>COUNT(*)</code>	Count all rows including NULLs	Includes NULLs
<code>COUNT(column)</code>	Count non-NULL values in column	Excludes NULLs
<code>SUM(column)</code>	Total sum of values	Ignores NULLs
<code>AVG(column)</code>	Average (sum/count of non-NULLs)	Ignores NULLs
<code>MIN(column)</code>	Smallest value	Ignores NULLs
<code>MAX(column)</code>	Largest value	Ignores NULLs

```
SELECT
  COUNT(*) as total_orders,
  COUNT(discount_code) as discounted_orders, -- only non-NULL discounts
  SUM(amount) as revenue,
  AVG(amount) as avg_order_value,
  MIN(order_date) as first_order,
```

```
MAX(order_date) as latest_order
FROM orders;
```

Real-World Example: An e-commerce CEO's dashboard query: `SELECT COUNT(*) as total_customers, AVG(total_spend) as avg_ltv, MAX(total_spend) as top_customer_spend FROM customer_summary WHERE signup_year = 2024`. At a glance: how many customers acquired in 2024, what's their average lifetime value, and who's the biggest spender — all from one aggregate query running in milliseconds on millions of rows.

Q84. What is the GROUP BY clause? How does it work?

Answer:

GROUP BY collapses rows with the same value in the specified column(s) into a single row — allowing aggregate functions to compute one value per group.

Rules:

- Every column in **SELECT** must either be in **GROUP BY** or inside an aggregate function
- Multiple columns in **GROUP BY** create groups for each unique combination

```
-- Sales by region and year
SELECT
  region,
  YEAR(sale_date) as year,
  SUM(amount) as total_sales,
  COUNT(*) as num_orders
FROM sales
GROUP BY region, YEAR(sale_date)
ORDER BY year, total_sales DESC;
```

Real-World Example: Zomato's analytics team runs: `SELECT city, restaurant_id, COUNT(*) as orders, AVG(rating) as avg_rating FROM orders GROUP BY city, restaurant_id HAVING COUNT(*) > 100`. This finds all restaurants with 100+ orders in each city and their average rating — powering the "Top Restaurants" feature. Without **GROUP BY**, they'd get one row per order (millions of rows) instead of one summary row per restaurant.

Q85. How do NULL values behave in SQL?

Answer:

NULL represents **unknown or missing data** — it's not zero, not empty string, not false. **NULL** has special behavior:

Operation	Result	Why
NULL = NULL	NULL (not TRUE!)	Can't compare unknowns

Operation	Result	Why
<code>NULL != NULL</code>	NULL	
<code>5 + NULL</code>	NULL	Any arithmetic with NULL = NULL
<code>NULL OR TRUE</code>	TRUE	Short-circuit evaluation
<code>NULL AND FALSE</code>	FALSE	
<code>COUNT(*)</code>	Counts NULLs	
<code>COUNT(col)</code>	Skips NULLs	
<code>AVG(col)</code>	Ignores NULLs	

How to check for NULL:

```
-- CORRECT
WHERE column IS NULL
WHERE column IS NOT NULL

-- WRONG (always returns NULL/false)
WHERE column = NULL
```

Real-World Example: A sales report calculates commission: `commission = salary * bonus_rate`. If `bonus_rate` is NULL for some employees (no bonus plan assigned), their commission becomes NULL — not 0. The report shows blank commission cells, confusing managers. Fix: `commission = salary * COALESCE(bonus_rate, 0)`. COALESCE is critical in any system where data can be missing — payroll systems, medical records, survey data.

PART D: Queries — Advanced

Q86. Explain different types of JOINS: INNER, LEFT, RIGHT, FULL OUTER.

Answer:

JOINS combine rows from two or more tables based on a related column.

JOIN Type	Returns
INNER JOIN	Only rows where the join condition matches in both tables
LEFT (OUTER) JOIN	All rows from left table + matched rows from right (NULL if no match)
RIGHT (OUTER) JOIN	All rows from right table + matched rows from left (NULL if no match)
FULL OUTER JOIN	All rows from both tables (NULL where no match on either side)
CROSS JOIN	Cartesian product — every row of left × every row of right

JOIN Type	Returns
SELF JOIN	Table joined with itself

```
-- INNER JOIN: Only customers who have placed orders
SELECT c.name, o.order_id FROM customers c
INNER JOIN orders o ON c.id = o.customer_id;

-- LEFT JOIN: ALL customers, including those with no orders
SELECT c.name, o.order_id FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;
-- Customers with no orders have NULL in order_id column
```

Real-World Example: A marketing team needs to find customers who **never placed an order** (to send a "We miss you" email): `SELECT c.* FROM customers c LEFT JOIN orders o ON c.id = o.customer_id WHERE o.order_id IS NULL`. The LEFT JOIN keeps all customers; `WHERE o.order_id IS NULL` filters to only those with no matching order — a classic "find rows with no corresponding record" pattern used in every CRM system.

Q87. What is a SELF JOIN? When would you use it?

Answer:

A **SELF JOIN** is when a table is joined **with itself** — treating the same table as two different tables using aliases. Used when a table has a column that references another row in the same table (hierarchical/recursive data).

```
-- Employees table: EmployeeID, Name, ManagerID (also an EmployeeID)
SELECT
    e.Name as Employee,
    m.Name as Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

Common use cases:

- Employee-Manager hierarchy
- Category-Subcategory trees
- Finding pairs (e.g., flights with same departure and destination)
- Bill of Materials (product components)

Real-World Example: A corporate org chart is stored in one `Employees` table with a `ManagerID` column pointing back to the same table's `EmployeeID`. A SELF JOIN retrieves each employee paired with their manager's name — enabling the HR system to display "Reports to: [Manager Name]" on every employee profile. LinkedIn's connection graph (finding mutual connections) also uses self-join-like queries on the `Connections(user_id, friend_id)` table.

Q88. What is the difference between UNION and UNION ALL?

Answer:

Both combine results from multiple SELECT statements. The columns and data types must match.

Feature	UNION	UNION ALL
Duplicates	Removes duplicate rows	Keeps all rows including duplicates
Performance	Slower (must sort/hash to find duplicates)	Faster (no deduplication)
Use when	Need distinct results	Need all results or know no duplicates exist
Memory	More (for deduplication)	Less

```
-- UNION: unique cities from both tables
SELECT city FROM customers
UNION
SELECT city FROM suppliers;

-- UNION ALL: all cities, duplicates included
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

Real-World Example: A bank's audit report combines transactions from two systems: a legacy `old_transactions` table and a new `new_transactions` table. Using `UNION ALL` is correct here — the two systems are separate sources with no overlap, and `UNION ALL` is 3× faster than `UNION`. But when generating a report of unique customer emails across `active_customers` and `trial_customers` tables (where a user might be in both), `UNION` correctly deduplicates — sending each person only one email.

Q89. Explain subqueries: correlated vs non-correlated.

Answer:

A **subquery** is a query nested inside another query.

Non-Correlated Subquery:

- Executes **once**, independently of the outer query
- Result is used by the outer query
- Like a constant value from the DB's perspective

```
-- Non-correlated: inner query runs once
SELECT name FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Correlated Subquery:

- **References the outer query** — runs once for **every row** of the outer query
- Slower (O(n) executions of inner query)
- Can be rewritten as JOIN for better performance

```
-- Correlated: inner query runs for each employee
SELECT e.name FROM employees e
WHERE e.salary > (
    SELECT AVG(salary) FROM employees
    WHERE department = e.department -- references outer query's row
);
```

Real-World Example: Finding employees earning above their own department's average (not company average) requires a **correlated subquery** — the inner query needs to know which department the current outer row belongs to. Each of 10,000 employees triggers the inner aggregate query — potentially slow. The optimization: rewrite as a JOIN with a precomputed department average CTE — runs the aggregate once per department, not once per employee.

Q90. What are Common Table Expressions (CTEs)?

Answer:

A **CTE** (defined with **WITH**) is a **named temporary result set** that exists only for the duration of the query. It makes complex queries more readable and maintainable.

```
-- CTE example
WITH department_averages AS (
    SELECT department_id, AVG(salary) as avg_sal
    FROM employees
    GROUP BY department_id
),
high_performers AS (
    SELECT e.*, da.avg_sal
    FROM employees e
    JOIN department_averages da ON e.department_id = da.department_id
    WHERE e.salary > da.avg_sal * 1.2 -- 20% above dept average
)
SELECT * FROM high_performers ORDER BY salary DESC;
```

Advantages:

- Improved readability (replace deeply nested subqueries)
- Reusable within the same query (reference multiple times)
- **Recursive CTEs** — traverse hierarchical data (org charts, graph paths)

Real-World Example: LinkedIn's "People you may know" might use a recursive CTE to find friends-of-friends: the CTE starts with your direct connections (level 1), then recursively finds their connections (level 2), filtering out people you already know. Recursive CTEs enable graph traversal directly in SQL — without application-level code — used in org chart displays, bill-of-materials explosions, and network analysis in SQL-based analytics platforms.

Q91. Explain window functions (ROW_NUMBER, RANK, DENSE_RANK).

Answer:

Window functions perform calculations across a **set of rows related to the current row** — without collapsing rows like GROUP BY. They "look through a window" at surrounding rows.

```
SELECT
    employee_id,
    name,
    department,
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) as row_num,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) as rank,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) as dense_rank
FROM employees;
```

Behavior for tied salaries (e.g., two employees with 50,000):

Function	Handles Ties
ROW_NUMBER()	Unique number for every row — tie-breaking is arbitrary
RANK()	Same rank for ties, skips next rank (1,1,3)
DENSE_RANK()	Same rank for ties, no gaps (1,1,2)

Other window functions: LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE(), NTILE(), SUM() OVER, AVG() OVER

Real-World Example: An e-commerce analytics report needs to find the **top 3 selling products per category**. Without window functions: complex self-joins. With window functions: `RANK() OVER (PARTITION BY category ORDER BY total_sales DESC) as sales_rank → WHERE sales_rank <= 3`. Amazon's internal analytics uses window functions extensively for category rankings, cohort analysis, and moving averages — queries that would require multiple passes or subqueries without them.

Q92. How do you find the Nth highest value in a table?

Answer:

Multiple approaches:

Method 1: Using DENSE_RANK (best — handles ties correctly)

```
-- Nth highest salary (e.g., N=3)
WITH ranked AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rnk
    FROM employees
)
SELECT DISTINCT salary FROM ranked WHERE rnk = 3;
```

Method 2: Using LIMIT/OFFSET

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET N-1; -- N=3: LIMIT 1 OFFSET 2
```

Method 3: Correlated subquery (classic interview answer)

```
SELECT MIN(salary) FROM employees
WHERE salary IN (
    SELECT DISTINCT TOP N salary FROM employees ORDER BY salary DESC
);
```

Real-World Example: A company's HR policy gives bonuses to the employee with the **3rd highest performance score** in each region. Using `DENSE_RANK() OVER (PARTITION BY region ORDER BY score DESC) = 3` correctly identifies them even when multiple employees tie for 3rd place — all tying employees get the bonus. The LIMIT/OFFSET approach fails here (gives only one person when there are ties), potentially causing unfair bonus distribution across regions.

PART E: DDL & DML

Q93. Explain CREATE, ALTER, DROP, and TRUNCATE.

Answer:

All are **DDL commands** that modify database structure.

Command	Purpose	Rollback
CREATE	Create new database object (table, index, view, procedure)	Auto-committed
ALTER	Modify existing object (add/drop/rename column, change data type)	Auto-committed
DROP	Permanently delete object and all its data	Auto-committed
TRUNCATE	Delete all rows from table; keep structure; faster than DELETE	Auto-committed (usually)

```
CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(100), gpa DECIMAL(3,2));

ALTER TABLE students ADD COLUMN email VARCHAR(200);
ALTER TABLE students RENAME COLUMN gpa TO grade_point;

TRUNCATE TABLE students;           -- removes all rows, keeps table structure

DROP TABLE students;               -- removes table entirely
```

Real-World Example: A startup launches a new feature — user profiles with profile pictures. The DBA runs `ALTER TABLE users ADD COLUMN profile_pic_url VARCHAR(500)` on the production database during low-traffic hours. This non-destructive change adds the column without losing existing user data. Contrast: during a data breach cleanup, they `DROP TABLE temp_export` to permanently remove a temporary table that had been exposed — no rollback possible, permanent deletion.

Q94. What is the difference between DELETE, TRUNCATE, and DROP?

Answer:

Feature	DELETE	TRUNCATE	DROP
What's removed	Specific rows	All rows	Entire table/object
Structure kept	✓ Yes	✓ Yes	✗ No
WHERE clause	✓ Supported	✗ Not supported	✗ Not supported
Rollback possible	✓ Yes (within transaction)	✗ Usually no (DDL)	✗ Usually no
Triggers fired	✓ Yes	✗ No	✗ No
Speed	Slow (row-by-row, logs each)	Fast (deallocates pages)	Instant
Auto-increment reset	✗ No	✓ Yes (usually)	—
Command type	DML	DDL	DDL

```
DELETE FROM orders WHERE order_date < '2020-01-01'; -- delete old orders, can
rollback

TRUNCATE TABLE temp_staging; -- clear staging table fast before reload

DROP TABLE old_archive; -- permanently remove old table
```

Real-World Example: A retail company's ETL pipeline clears a staging table before each nightly data load. Using `DELETE FROM staging` on 50 million rows: 8 minutes (logs each row deletion). Using `TRUNCATE TABLE staging`: 2 seconds (deallocates storage pages). TRUNCATE is the right choice for ETL workflows. But for GDPR compliance — deleting a specific customer's personal data — `DELETE FROM customers WHERE id = 12345` is the only option, with a transaction to ensure it can be rolled back if the audit system fails to log first.

Q95. Explain constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL.

Answer:

Constraints are rules enforced on table columns to ensure data integrity.

Constraint	Purpose	Example
PRIMARY KEY	Unique + Not Null — uniquely identifies each row	<code>id INT PRIMARY KEY</code>
FOREIGN KEY	Ensures referential integrity with another table	<code>FOREIGN KEY (dept_id) REFERENCES departments(id)</code>
UNIQUE	All values in column must be distinct (NULLs allowed)	<code>UNIQUE (email)</code>
NOT NULL	Column must have a value	<code>name VARCHAR(100) NOT NULL</code>
CHECK	Values must satisfy a condition	<code>CHECK (age >= 18)</code>
DEFAULT	Provides default value if none given	<code>status VARCHAR DEFAULT 'active'</code>

```
CREATE TABLE employees (  
  id INT PRIMARY KEY,  
  email VARCHAR(200) UNIQUE NOT NULL,  
  age INT CHECK (age BETWEEN 18 AND 70),  
  department_id INT FOREIGN KEY REFERENCES departments(id),  
  status VARCHAR(20) DEFAULT 'active' NOT NULL  
);
```

Real-World Example: A university student registration system uses all constraints together: `student_id` (PRIMARY KEY — uniqueness), `email` (UNIQUE — one email per student), `age` CHECK (`age >= 16`) (no underage enrollment), `program_id` (FOREIGN KEY — must be a valid program), `enrollment_status` NOT NULL (can't have unknown status). These constraints caught 47 data entry errors in the first semester after implementation, before bad data could corrupt reports.

PART F: Indexes & Performance

Q96. What are indexes? Why are they important?

Answer:

An **index** is a separate data structure (typically a B-Tree or Hash) that maintains a **sorted copy of specific column(s)** along with pointers to the actual row locations — enabling fast lookups without scanning the entire table.

Why important:

- Dramatically speed up **SELECT** queries with **WHERE**, **JOIN**, **ORDER BY**, **GROUP BY**
- Without index: **full table scan** $O(n)$
- With index: **B-Tree traversal** $O(\log n)$ or **hash lookup** $O(1)$

Trade-offs:

-  Faster reads
-  Slower writes (INSERT/UPDATE/DELETE must maintain the index)
-  Extra storage space

```
-- Without index: scans all 10 million rows
SELECT * FROM users WHERE email = 'alice@gmail.com';

-- Create index
CREATE INDEX idx_user_email ON users(email);

-- Now query uses index: finds row in ~20 comparisons (log2 10M)
```

Real-World Example: Twitter stores 500 million tweets/day. Without an index on **user_id**, loading someone's profile page requires scanning every tweet ever written — impossible at scale. The index on (**user_id**, **created_at**) makes **SELECT * FROM tweets WHERE user_id = 123 ORDER BY created_at DESC LIMIT 20** instantaneous — the B-Tree jumps directly to user 123's tweets. Indexing strategy is why Twitter's timeline loads in milliseconds despite petabytes of data.

Q97. Compare clustered vs non-clustered indexes.

Answer:

Feature	Clustered Index	Non-Clustered Index
Physical order	Table rows physically sorted by this index	Separate structure; rows in any order
Per table	Only one clustered index	Multiple (up to ~999)
Storage	No extra space (IS the table)	Extra structure stored separately
Speed (range queries)	Very fast (data already sorted)	Needs extra lookup to find rows
Speed (lookup by PK)	Fastest	Good (lookup + pointer follow)
Typical use	Primary key	Frequently queried non-PK columns

How non-clustered works: Stores index key + a pointer (row locator) to the actual data page. Two lookups: find the index entry, then follow the pointer to get the full row.

Real-World Example: A hospital patient database: **PatientID** (PRIMARY KEY) automatically creates a **clustered index** — patient records are physically stored in PatientID order. A doctor searches by **last_name** 100× more often than by ID. A **non-clustered index** on **last_name** stores (last_name → PatientID pointer).

The query jumps to the name in the index, follows the pointer to the clustered index to get the full record. SQL Server hospitals use this combination to make both PK lookups and name searches fast.

Q98. When should you create an index? When should you avoid it?

Answer:

Create an index when:

- Column appears frequently in **WHERE, JOIN, ORDER BY, GROUP BY**
- Column has **high cardinality** (many distinct values — e.g., email, SSN)
- Table has many rows (> 10,000) and queries need to return few rows
- Foreign key columns (for JOIN performance)

Avoid an index when:

- Table is small (full scan is faster than index overhead)
- Column has **low cardinality** (few distinct values — e.g., gender: M/F/Other — index barely helps)
- Column is frequently updated (index must be rebuilt often)
- Table has heavy INSERT/UPDATE/DELETE load (indexes slow down writes)
- Column is rarely used in queries

Real-World Example: An e-commerce product table with 2 million products: **CREATE INDEX idx_product_category ON products(category_id)** dramatically speeds up "show all electronics" queries. But **CREATE INDEX idx_product_in_stock ON products(in_stock)** where in_stock is just true/false is wasteful — only 2 distinct values means 50% of rows match, and the database optimizer will use a full scan anyway. Query Execution Plan analysis (EXPLAIN in MySQL/PostgreSQL) reveals when indexes are actually used vs. ignored by the optimizer.

PART G: Transactions

Q99. What is a transaction? Explain ACID properties.

Answer:

A **transaction** is a sequence of one or more SQL operations treated as a **single unit of work** — either all succeed completely or all fail completely.

ACID Properties:

Property	Meaning	Example
Atomicity	All operations succeed, or none do — "all or nothing"	Transfer: debit + credit both succeed or both fail
Consistency	Transaction brings database from one valid state to another — no constraints violated	Balance never goes negative

Property	Meaning	Example
Isolation	Concurrent transactions don't interfere with each other	Two transfers don't corrupt each other
Durability	Committed transactions persist even after crashes	Power failure after COMMIT → transaction not lost

```

BEGIN TRANSACTION;
    UPDATE accounts SET balance = balance - 5000 WHERE id = 1; -- debit
    UPDATE accounts SET balance = balance + 5000 WHERE id = 2; -- credit
COMMIT;
-- If anything fails between BEGIN and COMMIT:
ROLLBACK; -- both updates reversed

```

Real-World Example: HDFC Bank processes 10 million transactions daily. Without ACID: a power failure between the debit and credit in a fund transfer could leave money "in transit" — debited from sender, never credited to receiver. ACID guarantees: if the server crashes after debit but before credit, ROLLBACK restores the sender's balance on recovery. The bank's reputation (and RBI compliance) depends on ACID guarantees being upheld at every transaction.

Q100. Explain isolation levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE.

Answer:

Isolation levels control how much one transaction is **isolated from the effects of other concurrent transactions**. Higher isolation = more consistency but less performance (more locking).

Problems isolation levels prevent:

Problem	Description			
Dirty Read	Reading uncommitted changes from another transaction			
Non-repeatable Read	Same query returns different values within one transaction			
Phantom Read	New rows appear in repeated queries due to another transaction's INSERT			
Isolation Level	Dirty Read	Non-repeatable	Phantom Read	Use Case
READ UNCOMMITTED	❌ Possible	❌ Possible	❌ Possible	Fastest; for approximate stats only
READ COMMITTED	✅ Prevented	❌ Possible	❌ Possible	Default in most DBs (Oracle, PostgreSQL)
REPEATABLE READ	✅ Prevented	✅ Prevented	❌ Possible	Default in MySQL InnoDB

Isolation Level	Dirty Read	Non-repeatable	Phantom Read	Use Case
SERIALIZABLE	✓ Prevented	✓ Prevented	✓ Prevented	Strictest; financial transactions

Real-World Example: An airline seat booking system uses **SERIALIZABLE** isolation: when two users try to book the last seat on the same flight simultaneously, serializable isolation ensures they run serially — one books the seat, the other gets "seat no longer available." With **READ COMMITTED**, both might read "1 seat available," both try to book, and the database oversells — an expensive real-world mistake that happened to major airlines before proper isolation was implemented.

QUICK REVISION CHEAT SHEET

Java Quick Reference

Topic	Key Point
OOP Pillars	Encapsulation, Inheritance, Polymorphism, Abstraction
JVM/JRE/JDK	Engine / Runtime / Full Dev Kit
Pass-by-value	Always copies — objects pass copy of reference
HashMap	Array of buckets + chaining; O(1) avg lookup
Generics	Type safety at compile time
Lambda	(x) -> x*2 — anonymous function for functional interfaces
Stream API	Declarative data processing; lazy evaluation
GC	Young/Old generation; automatic memory management

Python Quick Reference

Topic	Key Point
GIL	One thread at a time; use multiprocessing for CPU tasks
Mutable	list, dict, set — changeable
Immutable	int, str, tuple — unchangeable, hashable
Generators	yield — lazy evaluation, memory efficient
Decorators	Function wrapping — @timer, @login_required
Context Manager	with statement — guaranteed cleanup
List comprehension	[x**2 for x in range(n)] — concise + fast
Deep vs Shallow copy	copy.deepcopy() for truly independent copies

Data Structures Quick Reference

Structure	Insert	Delete	Search	Use Case
Array	$O(n)$ mid	$O(n)$ mid	$O(1)$ index	Random access
Linked List	$O(1)$	$O(1)$	$O(n)$	Dynamic insertions
Stack	$O(1)$	$O(1)$	$O(n)$	Undo, DFS
Queue	$O(1)$	$O(1)$	$O(n)$	BFS, scheduling
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	Fast lookup
BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	Ordered data
Heap	$O(\log n)$	$O(\log n)$	$O(1)$ min/max	Priority queue

SQL Quick Reference

Concept	Key Point
ACID	Atomicity, Consistency, Isolation, Durability
JOIN types	INNER=match only; LEFT=all left; FULL=all rows
WHERE vs HAVING	WHERE=rows before grouping; HAVING=groups after
NULL	IS NULL, never = NULL; propagates in math
CTE	WITH name AS (...) — readable temporary result set
Window functions	ROW_NUMBER, RANK, DENSE_RANK — no row collapsing
Index	B-Tree for fast lookup; slows writes; high cardinality columns
Normalization	1NF=atomic; 2NF=no partial dep; 3NF=no transitive dep